
Machine Learning

Universidad Santo Tomás

Maestría en Liderazgo y Gestión Estratégica de la
Transformación Digital

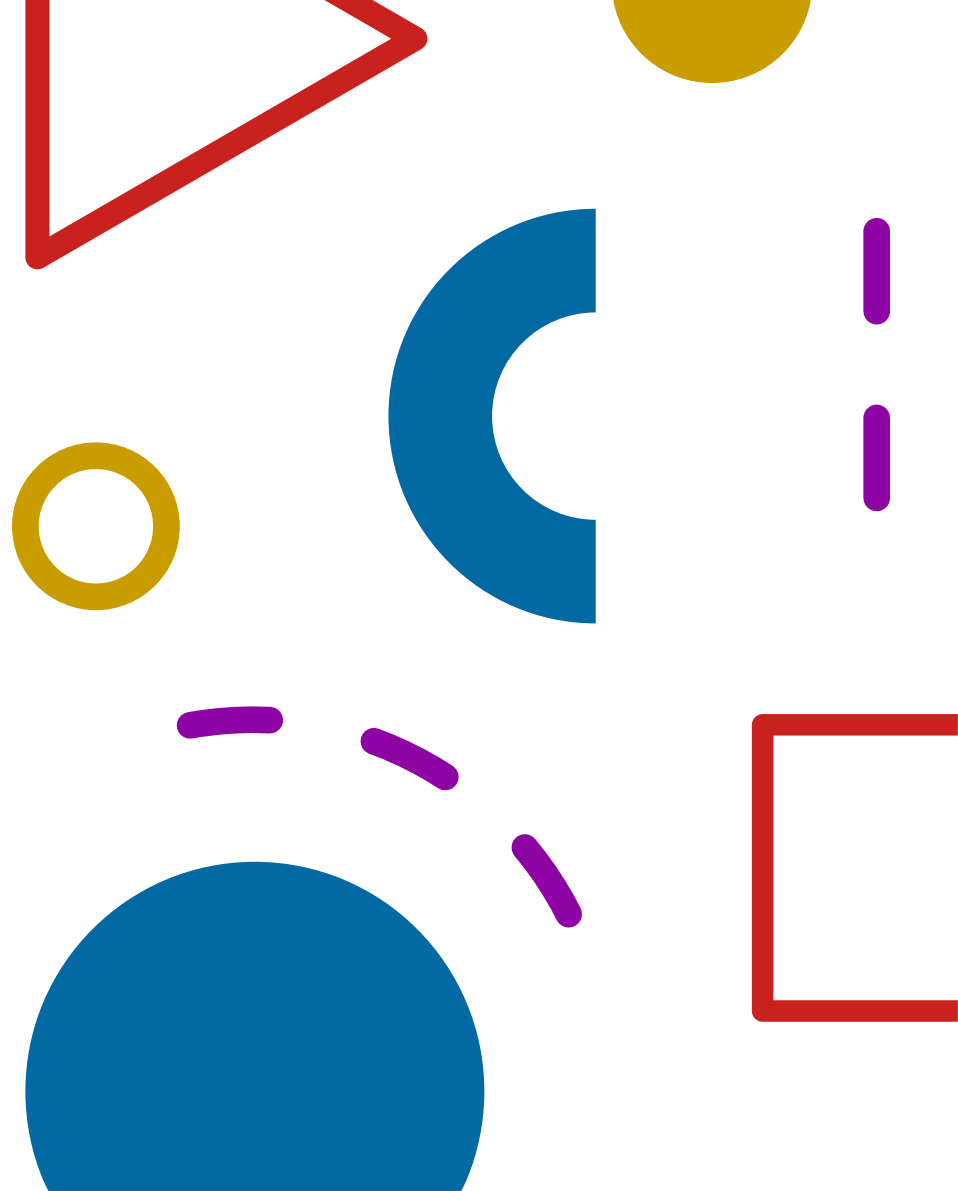
Entornos de Transformación Digital – **Machine Learning**

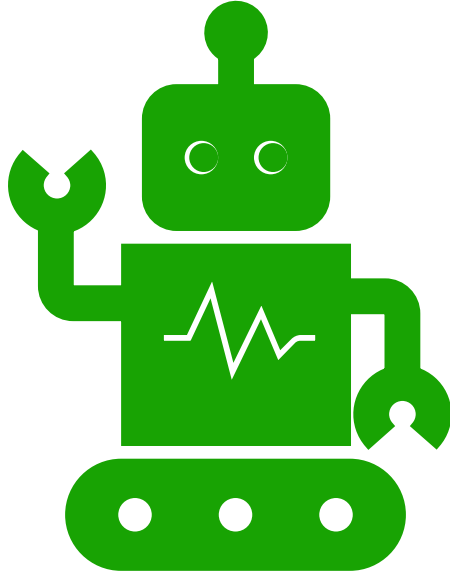
Ing. Luz Santamaría Granados, PhD.

2026



Generalidades de Machine Learning





¿Qué es Machine Learning?

- El aprendizaje automático es una forma de enseñar a los computadores a tomar decisiones o hacer predicciones aprendiendo de los datos, en lugar de seguir reglas explícitas escritas por un programador.
- El aprendizaje automático no sigue reglas predefinidas.
- Descubre patrones en los datos automáticamente; cuantos más datos de calidad se le proporcionen, mejor aprenderá.

Generalidades de Machine Learning



¿Qué es un sistema de Machine Learning (ML)?



¿Cuándo tiene sentido utilizar ML?

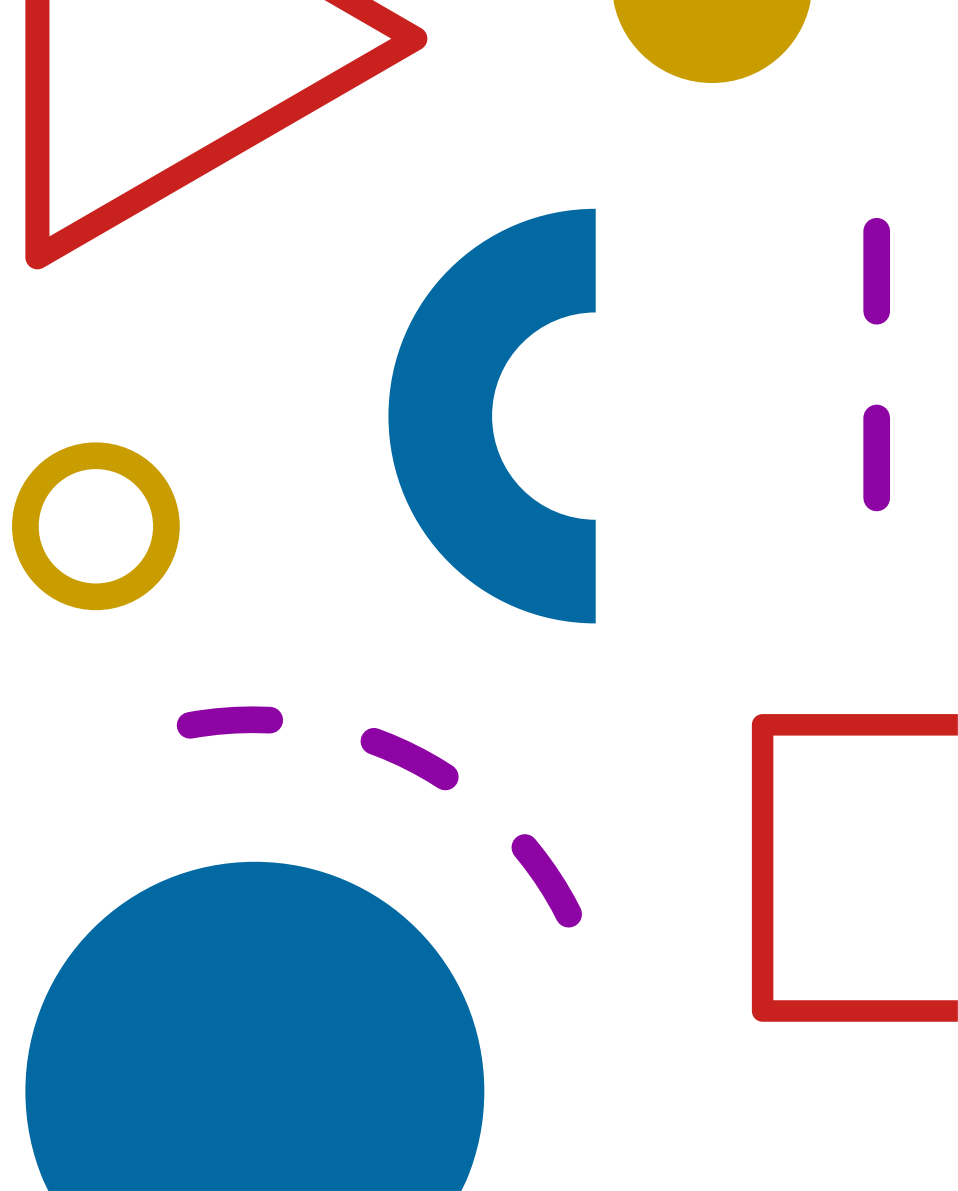


¿Qué condiciones debe cumplir un problema?



ML no es solo un algoritmo: es un sistema completo.

Esquema de un Sistema de Machine Learning

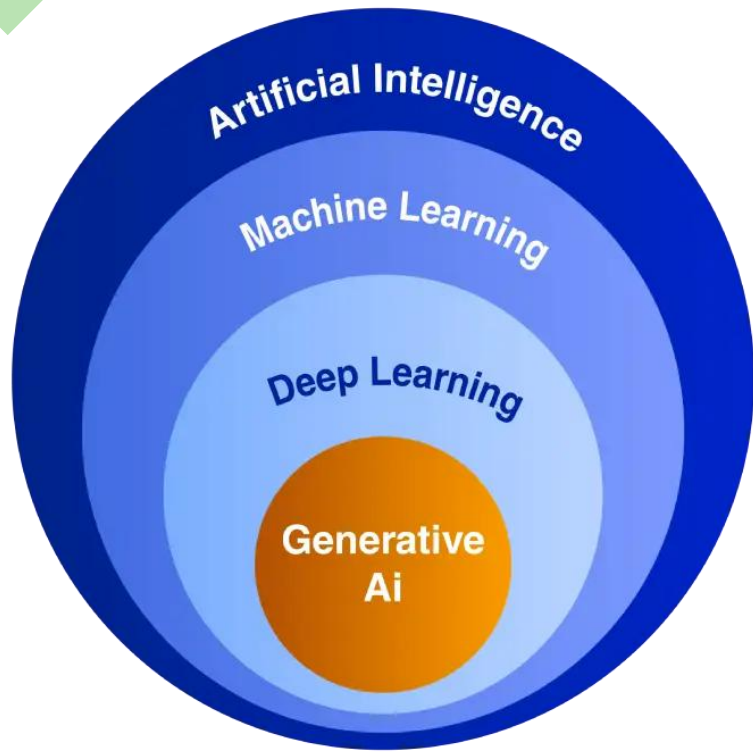


El auge del Machine Learning

Caso Google Translate — 2016

Google incorporó su sistema de traducción automática neuronal multilingüe a Google Translate.

- El uso de redes neuronales profundas produjo una mejora significativa en la calidad de las traducciones.
- Este éxito impulsó nuevamente el interés por el Deep Learning.
- Desde entonces, ML se ha extendido a múltiples áreas:
 - Comunicación
 - Búsqueda de información
 - Trabajo
 - Recomendaciones
 - Salud
 - Transporte
 - Agricultura
- El ML pasó de ser una tecnología especializada para formar parte de numerosos productos y servicios cotidianos.



Point of Difference	Generative AI	Machine Learning	Deep Learning	LLMs
Focus	Creating new content autonomously	Learning patterns from data	Learning complex patterns using multi-layer neural networks	Language-related tasks
Core Functions	Generates new content based on learned patterns	Analyzes data to make predictions or decisions	Learns complex representations for accurate predictions	Processes and generates text from extensive training data
Key Algorithms	GANs, VAEs, Diffusion Models	Decision Trees, SVMs, Random Forests, Naive Bayes	CNNs, RNNs, Transformers	Transformer-based models such as GPT, BERT, T5, and variants
Application	Text generation, image synthesis, music creation, drug discovery	Spam detection, credit scoring, recommender systems, predictive maintenance	Computer vision, NLP, speech recognition, autonomous vehicles	Chatbots, virtual assistants, content generation, translation
Complexity Area	Probabilistic models and algorithms for content generation	Algorithms such as decision trees, SVMs, and neural networks	Intricate neural network architectures with multiple layers	Sophisticated language models trained on large datasets

Un algoritmo no es un sistema de ML

Un sistema de ML incluye:

- Requisitos del negocio
- Usuarios e interfaces
- Datos
- Algoritmos y modelos
- Lógica de desarrollo y actualización
- Monitoreo
- Infraestructura

¿Cuándo utilizar Machine Learning?

ML no es una solución mágica.

Antes de comenzar un proyecto debemos preguntarnos:

- ¿Realmente necesitamos ML?
- ¿Existe una alternativa más sencilla?
- ¿Es rentable desarrollar y mantener el sistema?
- ¿Tenemos suficientes datos?
- ¿El problema requiere predicciones?

Ejemplo:

-  Determinar el estado de una propiedad a partir de un código postal → tabla de consulta.
-  Predecir el precio de alquiler → ML puede ser apropiado.

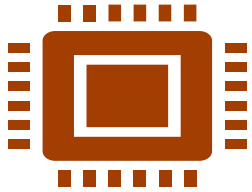
¿Qué hace realmente un sistema de ML?

Aprender patrones complejos a partir de datos existentes para realizar predicciones sobre datos que no ha visto.

Condiciones importantes:

- Repetición
- Bajo costo de errores
- Escala
- Patrones cambiantes

1. Aprender



Un sistema ML debe tener capacidad de aprender a partir de datos.

Ejemplo: [Airbnb](#)

Entradas:

- Superficie
- Número de habitaciones
- Ubicación
- Servicios
- Calificación

Salida:

- Precio de alquiler

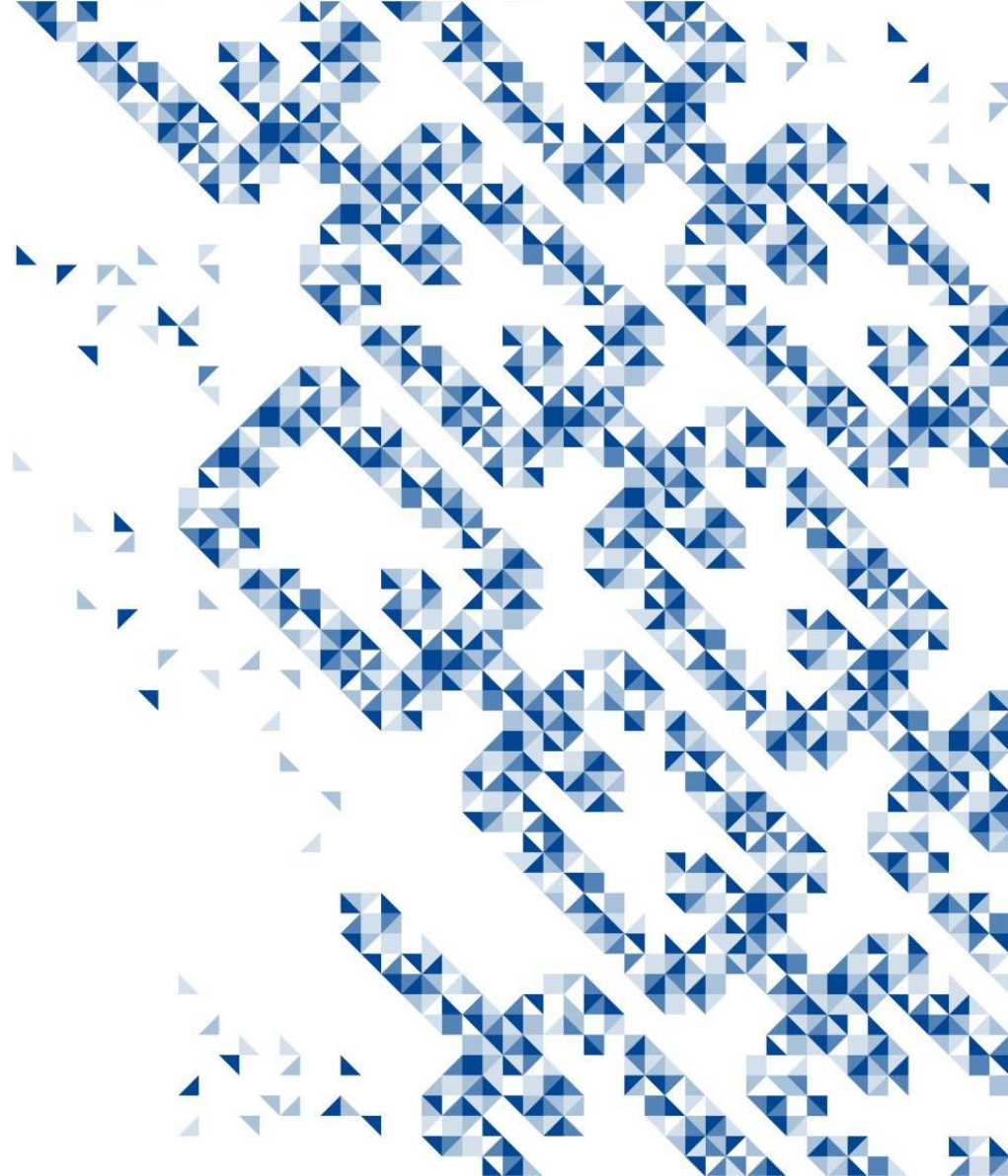


El modelo aprende la relación entre las características y el precio.

2. Patrones complejos

ML es especialmente útil cuando la relación entre las variables es difícil de definir mediante reglas manuales.

- Regla simple → software tradicional
- **Patrón complejo → ML**



Software tradicional vs. Machine Learning

Software tradicional	Machine Learning
El programador define las reglas	El sistema aprende patrones
Reglas explícitas	Datos + entrenamiento
Adecuado para problemas simples	Adecuado para relaciones complejas
Fácil de especificar manualmente	Difícil de expresar con reglas

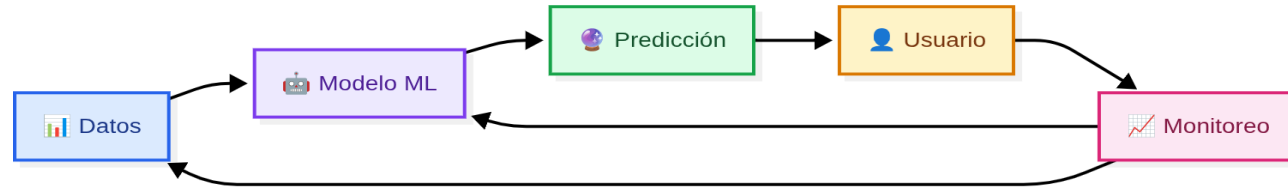
3. Datos existentes

- Sin datos no hay aprendizaje tradicional.

- Para entrenar un modelo necesitamos:

- Ejemplo: Para predecir impuestos necesitamos información sobre:

- Ingresos
- Características de las personas
- Historial fiscal
- Población suficientemente grande



Excepciones y alternativas



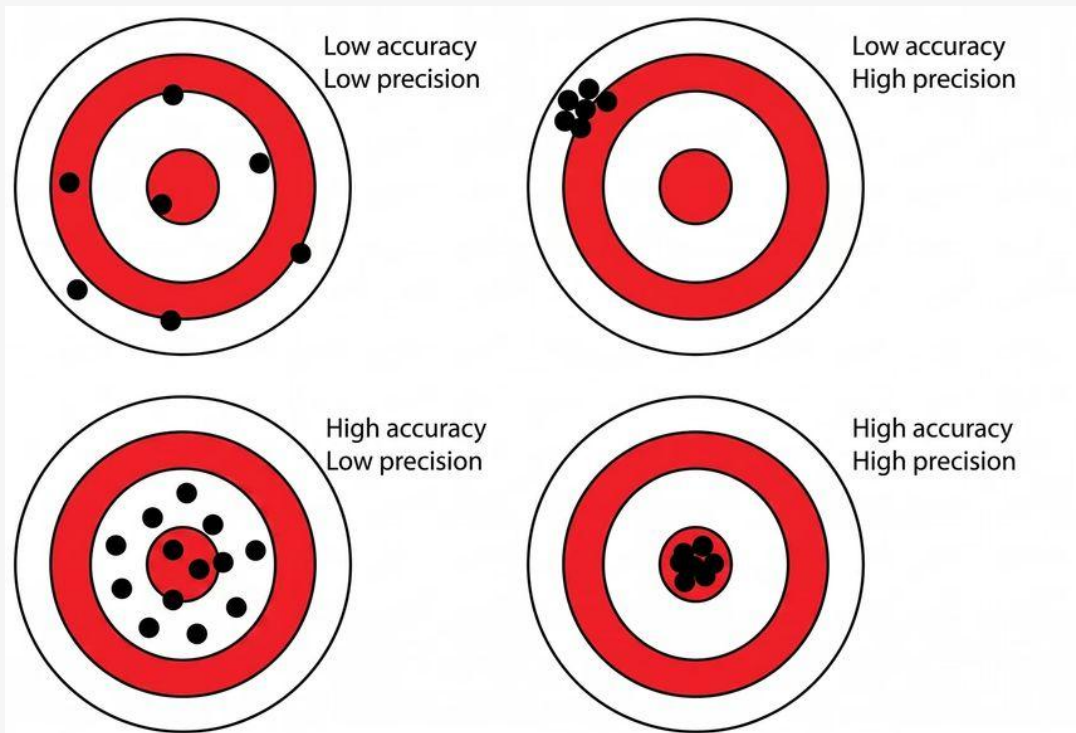
Zero-shot (Arranque en frío) learning: el modelo puede realizar una tarea sin datos específicos de esa tarea, pero fue entrenado previamente con otros datos.



Continual learning: el modelo puede aprender de datos que llegan durante la producción.



“Fake it till you make it” (*Finge hasta que lo logres*): inicialmente las predicciones pueden ser realizadas por humanos para generar datos que posteriormente permitan entrenar modelos.



4. Predicciones

- ML debe responder a preguntas que puedan formularse como: **¿Cuál sería el resultado?**
- Ejemplos:
 - ¿Qué película verá un usuario?
 - ¿Qué correo es spam?
 - ¿Cuál será el precio de una vivienda?
 - ¿Cómo se verá una imagen después de eliminar ruido?
- Problemas computacionalmente costosos pueden reformularse como problemas de predicción aproximada.

5. Datos no vistos

El modelo debe funcionar con datos nuevos que compartan patrones con los datos de entrenamiento.



Entrenamiento $\approx$
realidad futura



Si la distribución de los datos cambia demasiado, el modelo puede perder rendimiento.

¿Qué características favorecen el uso de ML?

Un problema es especialmente adecuado cuando:

- **Es repetitivo:** Hay muchos ejemplos del mismo patrón.
- **El costo de equivocarse es relativamente bajo:**
 - Ejemplo: una mala recomendación normalmente solo implica que el usuario no haga clic.
- **Está a escala:** Millones de predicciones justifican la inversión en datos e infraestructura.
- **Los patrones cambian:** ML puede actualizarse con nuevos datos, mientras que las reglas manuales pueden quedar obsoletas.

Ejemplos y límites

Buenos candidatos para ML

- Sistemas de recomendación
- Reconocimiento de voz
- Detección de objetos
- Clasificación de spam
- Predicción de precios
- Enrutamiento de solicitudes
- Traducción automática

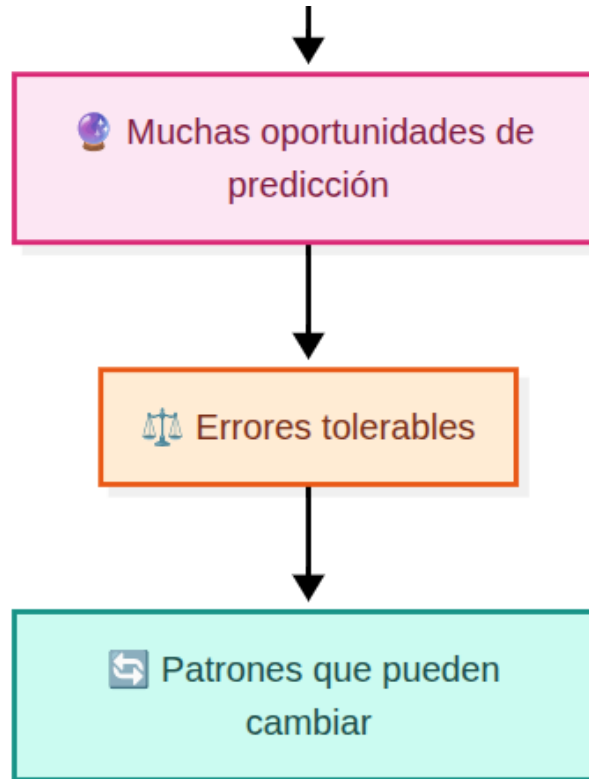
Impacto de los errores en ML

El costo de los errores importa.

- Recomendación incorrecta: generalmente bajo impacto.
- Error en conducción autónoma: potencialmente consecuencias graves.



Por eso, no basta con que ML sea técnicamente posible: también debe ser seguro, útil y económicamente viable.



Características
de los
Problemas con
ML

Modelos de análisis

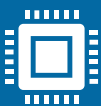
Diferencias entre los distintos tipos de modelos de análisis



PREDIK
Data-Driven

Tomado de [Predik Data-Driven](https://predik.com)

Consideraciones de ML



Machine Learning **no** consiste simplemente en elegir un algoritmo. Es construir un sistema capaz de aprender, predecir, adaptarse y funcionar de manera confiable en producción.



Para un líder empresarial la pregunta no es ¿cuál es el algoritmo más sofisticado?, sino ¿qué problema estratégico puede resolverse mediante ML?

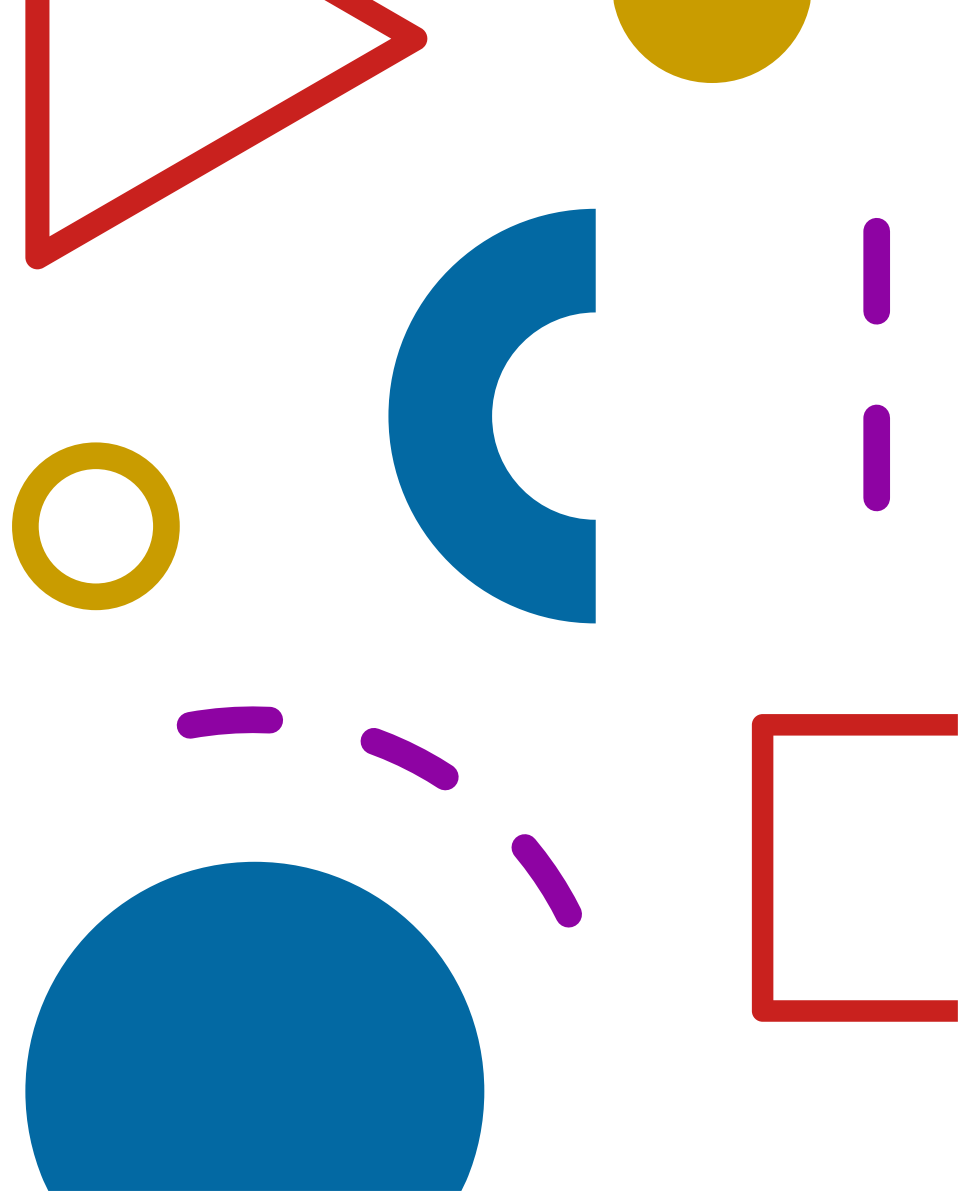


No siempre debemos utilizar ML, pero tampoco debemos descartarlo demasiado pronto: una tecnología que hoy no sea la opción más eficiente puede volverse fundamental a medida que evolucionan los datos, el hardware y las herramientas.

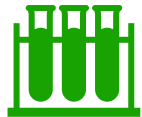
Machine Learning Statistics 2024



Casos de uso del Machine Learning



Casos de Uso de ML



El ML está presente tanto en aplicaciones de consumo como empresariales.



Desde mediados de la década de 2010, su adopción ha crecido rápidamente.



Permite:

- Mejorar servicios existentes.
- Automatizar tareas.
- Realizar predicciones.
- Hacer posibles servicios que antes eran difíciles o imposibles.



ML ya no es una tecnología experimental: forma parte de productos y procesos cotidianos.

ML en nuestra vida cotidiana

Aplicaciones de consumo

-  Búsqueda
 - Motores de búsqueda utilizan ML para ordenar resultados.
-  Recomendaciones
 - Amazon, Netflix y otras plataformas predicen qué puede interesarnos.
-  Texto predictivo
 - El teléfono sugiere palabras o frases mientras escribimos.
-  Edición de fotografías
 - ML puede recomendar mejoras para nuestras imágenes.
-  Autenticación
 - Reconocimiento facial y de huellas digitales.

ML para comunicación y hogares inteligentes

Traducción automática

- Traduce texto y voz entre diferentes idiomas.
- Reduce las barreras lingüísticas.
- Facilita la comunicación entre personas de diferentes culturas.

Hogares inteligentes

- Asistentes personales.
- Cámaras de seguridad inteligentes.
- Detección de actividades inusuales.
- Monitoreo de personas mayores.
- Detección de caídas.

Ejemplo:

- Un sistema puede predecir si una persona ha sufrido una caída y generar una alerta.

ML de consumo vs ML empresarial

Aplicaciones de consumo	Aplicaciones empresariales
Orientadas al usuario final	Orientadas a procesos y negocio
Facilidad de distribución importante	Integración con sistemas existentes
Latencia muy importante	Puede tolerarse mayor latencia
Monetización más difícil	ROI y ahorro pueden ser muy altos
Ej.: recomendaciones	Ej.: optimización de recursos

ML en empresas: ¿para qué se utiliza?

- . Uso interno
 - Reducir costos.
 - Automatizar procesos.
 - Generar información sobre clientes.
 - Mejorar la toma de decisiones.
 - . Uso externo
 - Mejorar la experiencia del cliente.
 - Retener clientes.
 - Interactuar con clientes.
 - Personalizar productos y servicios.
-
- eficiencia + clientes + ingresos

Detección de fraude

- Uno de los usos empresariales más antiguos de ML.
- Problema: Las transacciones pueden ser fraudulentas.
- Solución: El ML aprende a partir de transacciones históricas:
- Técnica relacionada: Detección de anomalías
- Objetivo: identificar comportamientos que se alejan de los patrones normales.



Optimización de precios

Price Optimization

- Determinar el precio adecuado en un momento determinado para maximizar un objetivo.
- Puede buscar maximizar:
 - 💰 Margen
 - 📈 Ingresos
 - 🚀 Crecimiento
- ¿Cuándo es especialmente útil?
- Cuando existe:
 - Gran cantidad de transacciones.
 - Demanda variable.
 - Clientes dispuestos a aceptar precios dinámicos.

Ejemplos de Optimización de Procesos



✈ VUELOS



🏨 HOTELES



🚗
TRANSPORTE
COMPARTIDO



📢 PUBLICIDAD
ONLINE



🎫 EVENTOS

Predicción de demanda

- Las empresas necesitan anticipar cuánto demandarán sus productos o servicios.
 - Permite:
 - Planificar presupuestos.
 - Gestionar inventarios.
 - Asignar recursos.
 - Ajustar precios.
 - Ejemplo: supermercado
 - Demanda demasiado baja → exceso de inventario → productos desperdiciados.
 - Demanda demasiado alta → falta de productos → ventas perdidas.
 - ML → predicción de demanda → mejor planificación

Adquisición de clientes

- Conseguir nuevos clientes puede ser muy costoso.
- ML puede ayudar a reducir este costo mediante:
 - 🎯 Identificación de clientes potenciales
 - 🗂️ Publicidad más segmentada
 - 💰 Descuentos en el momento adecuado
 - 📊 Predicción de probabilidad de conversión

Resultado

- Mejor selección de clientes → menor costo de adquisición → mayor rentabilidad
- Incluso una pequeña reducción en el costo de adquisición puede tener un impacto significativo en las ganancias.



Predicción de abandono (Churn)

- ¿Qué clientes tienen mayor probabilidad de dejar de utilizar nuestro producto o servicio?
 - ML puede identificar:
 - Señales de abandono.
 - Clientes con alto riesgo.
 - Momento probable de abandono.
 - ¿Qué puede hacer la empresa?
 - Una vez identificado el riesgo:
 - → Contactar al cliente
 - → Ofrecer ayuda
 - → Personalizar una oferta
 - → Resolver problemas
 - → Intentar retenerlo
 - También puede aplicarse a empleados, para detectar riesgo de rotación.

Automatización y monitoreo de marca



Clasificación automática de tickets

- ML analiza el contenido de una solicitud y predice:
- ¿A qué departamento debe enviarse?
 - Ticket → ML → Clasificación → Departamento adecuado
- Beneficios:
 - Menor tiempo de respuesta.
 - Menos trabajo manual.
 - Mayor satisfacción del cliente.

Automatización y monitoreo de marca

-
-  **Brand Monitoring**
 - ML puede detectar:
 - . Dónde se menciona una marca.
 - . Cómo se menciona.
 - . Referencias directas e indirectas.
 - . Sentimiento positivo o negativo.
 - Ejemplo:
 - Un aumento repentino de comentarios negativos puede activar una respuesta rápida.

Machine Learning en salud

- Uno de los campos con mayor potencial y mayores exigencias.
 - Aplicaciones:
 -  Detección de enfermedades
 - Cáncer de piel.
 - Diabetes.
 -  Asistencia al diagnóstico
 - Apoyo a médicos.
 - Análisis de información médica.

Machine Learning en salud

¿Por qué es diferente?

Los sistemas de salud tienen requisitos especialmente estrictos en:

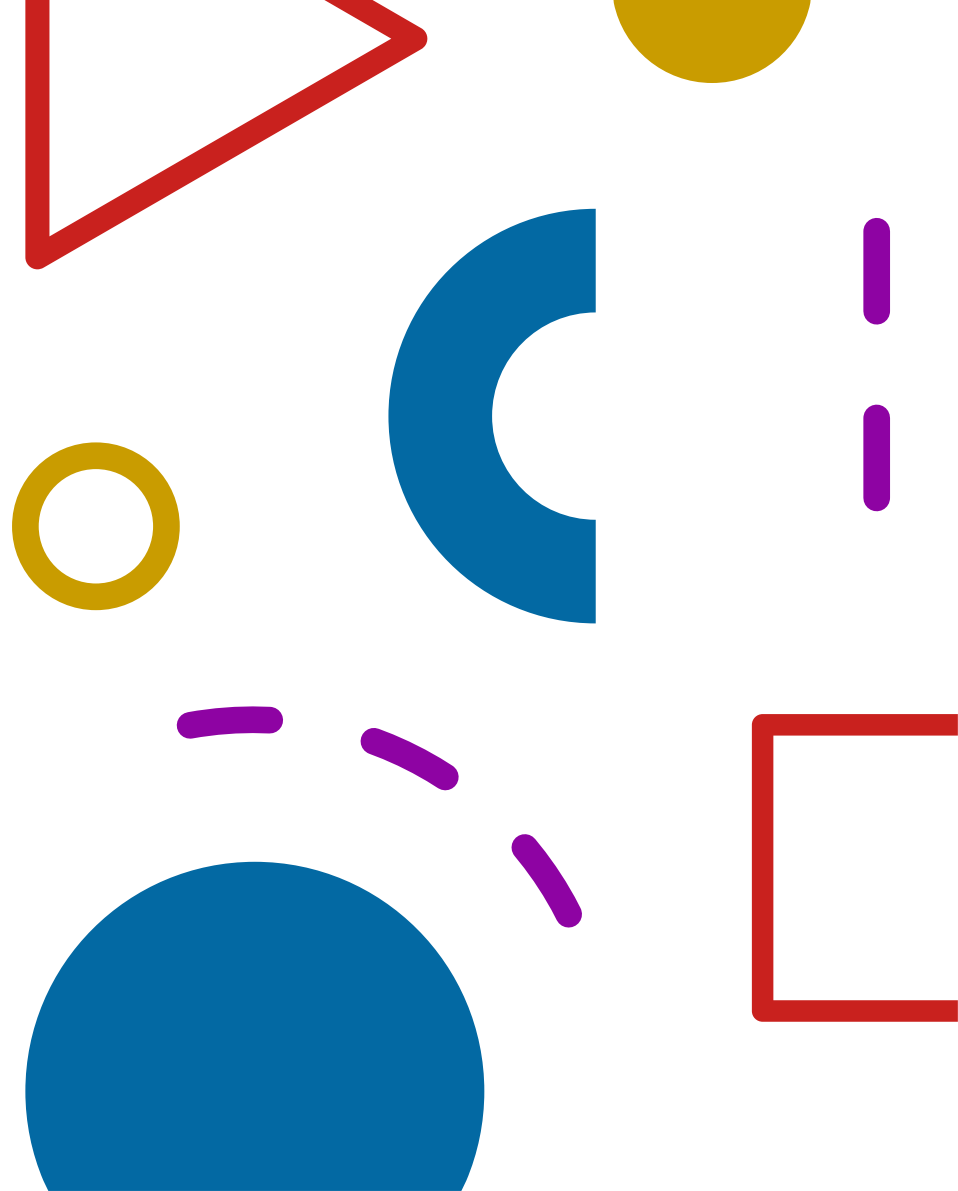
- Precisión
- Privacidad
- Seguridad
- Responsabilidad

Por ello, muchas soluciones de ML funcionan como herramientas de apoyo a profesionales sanitarios, en lugar de sustituirlos.

Los casos de uso de ML abarcan desde recomendar una película hasta detectar fraude, optimizar precios, predecir abandono y ayudar en diagnósticos médicos.

El valor del ML depende de adaptar la tecnología a las necesidades, riesgos y objetivos de cada contexto.

Comprendiendo los sistemas de Machine Learning



Comprendiendo los sistemas de Machine Learning

- Diseñar un sistema ML requiere comprender mucho más que los algoritmos.
- ML en investigación y ML en producción tienen objetivos diferentes.
- Los sistemas ML presentan desafíos particulares frente al software tradicional.
- ¿Qué cambia al pasar de investigación a producción?
- ¿Qué desafíos aparecen con datos, justicia e interpretabilidad?
- ¿Por qué ML necesita prácticas adicionales a las del software tradicional?

Investigación vs. producción

Aspecto	Investigación	Producción
Objetivo	Mejor rendimiento en benchmarks	Cumplir múltiples requisitos
Cómputo	Entrenamiento rápido	Inferencia rápida
Rendimiento	Alto throughput (rendimiento)	Baja latencia
Datos	Generalmente estáticos	Cambian constantemente
Fairness (justicia)	Puede no ser prioritaria	Debe considerarse
Interpretabilidad	Puede no ser prioritaria	Frecuentemente necesaria

Diferentes stakeholders, diferentes objetivos

- Un sistema ML en producción involucra múltiples equipos.
- Ejemplo: app que recomienda restaurantes
 -  Ingenieros ML → quieren mejores predicciones.
 -  Ventas → quieren restaurantes que generen mayores ingresos.
 -  Producto → quiere recomendaciones en menos de 100 ms.
 -  Plataforma ML → quiere estabilidad y escalabilidad.
 -  Gerencia → quiere maximizar el margen.

No existe un único objetivo

- Supongamos dos modelos:
 - Modelo A: Predice los restaurantes que el usuario tiene más probabilidades de elegir.
 - Modelo B: Predice los restaurantes que generan más ingresos para la empresa.
 - Pero además: Producto exige 🕒 Latencia < 100 ms.
 - Si A y B tardan más de 100 ms: ❌ Ninguno cumple los requisitos.
 - Antes de elegir un modelo debemos conocer:
 - Quiénes son los stakeholders.
 - Qué objetivos tienen.
 - Qué requisitos son obligatorios.
 - Qué requisitos son deseables.

Investigación $\neq$ producción

Una técnica puede funcionar muy bien en una competición y no ser adecuada para producción.

Ejemplo: Ensembling

Combina varios modelos para mejorar el rendimiento.

- 🏆 En investigación:
 - Puede mejorar ligeramente la métrica.
- 🏭 En producción:
 - Mayor complejidad.
 - Mayor costo computacional.
 - Mayor latencia.
 - Puede dificultar la interpretación.

Una mejora pequeña en una métrica solo vale la pena si justifica el costo y la complejidad adicionales.

Prioridades computacionales

- Durante el desarrollo: Entrenar → entrenar → entrenar → evaluar
 - ➡ El cuello de botella suele ser el entrenamiento.
- En producción: Solicitud → modelo → predicción → respuesta
 - ➡ El cuello de botella suele ser la inferencia.
- Investigación
 - ⚡ Entrenamiento rápido
 - 📊 Alto throughput
- Producción
 - ⚡ Inferencia rápida
 - ⌚ Baja latencia

Latencia vs. Throughput

Latencia: tiempo que tarda una solicitud en recibir respuesta.

Throughput: cantidad de solicitudes procesadas durante un período.

Ejemplo sencillo

- Si cada consulta tarda 10 ms: → aproximadamente 100 consultas/segundo si se procesan una a una.
- Si tarda 100 ms: → aproximadamente 10 consultas/segundo.
- Los sistemas modernos pueden procesar consultas en **lotes** (batches).
- Esto puede aumentar el throughput, aunque esperar a formar un lote puede aumentar la latencia.
- Trade-off: Más batching → potencialmente más throughput, pero también mayor latencia.

+

•

○

La latencia es una distribución

- No debemos mirar solamente la latencia promedio.
- Ejemplo:
 - 9 solicitudes $\approx$ 100 ms
 - 1 solicitud = 3.000 ms
- El promedio puede parecer 390 ms, aunque la mayoría de las solicitudes son mucho más rápidas.
 - Mejor enfoque: percentiles
 - p50 $\rightarrow$ mediana
 - p90 $\rightarrow$ 90% de solicitudes están por debajo de este valor
 - p95 $\rightarrow$ 95%
 - p99 $\rightarrow$ 99%
 - Los percentiles altos permiten detectar outliers y problemas que afectan a una parte de los usuarios.
 - Ejemplo de requisito: p99 de latencia $<$ 200 ms.

Los datos: investigación vs. producción

En **investigación**, los datasets suelen ser:

- Limpios.
- Bien estructurados.
- Estáticos.
- Conocidos.
- Preparados para benchmarking.

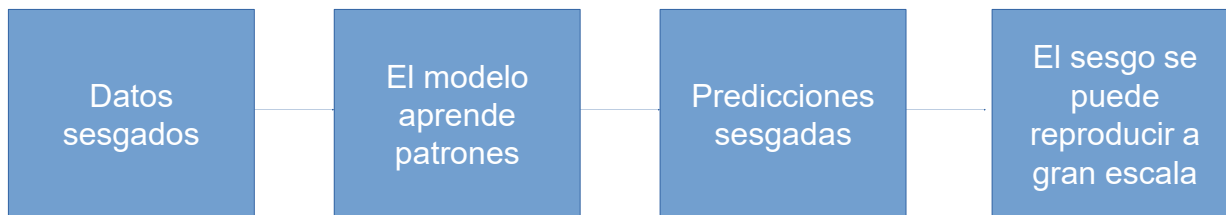
En **producción**, los datos suelen ser:

- Ruidosos.
- Incompletos.
- No estructurados.
- Cambiantes.
- Sesgados.
- Con etiquetas incorrectas o escasas.

Fairness: el modelo también puede ser injusto



- Los modelos ML aprenden de datos históricos. Si los datos contienen sesgos:



- Ejemplos de posibles consecuencias
 - Solicitudes de crédito.
 - Selección de candidatos.
 - Tasas hipotecarias.
 - Sistemas de vigilancia.
 - Rankings universitarios.

Interpretabilidad

- ¿Podemos entender por qué el modelo tomó una determinada decisión?
- Esto es especialmente importante cuando las decisiones afectan a personas.
- Ejemplo conceptual:
 -  IA → recomienda un tratamiento médico.
 - ¿El médico y el paciente deberían poder entender por qué?

La interpretabilidad ayuda a:

- Generar confianza.
- Detectar sesgos.
- Explicar decisiones.
- Depurar modelos.
- Mejorar el sistema.

ML: Código + datos

En ML necesitamos:

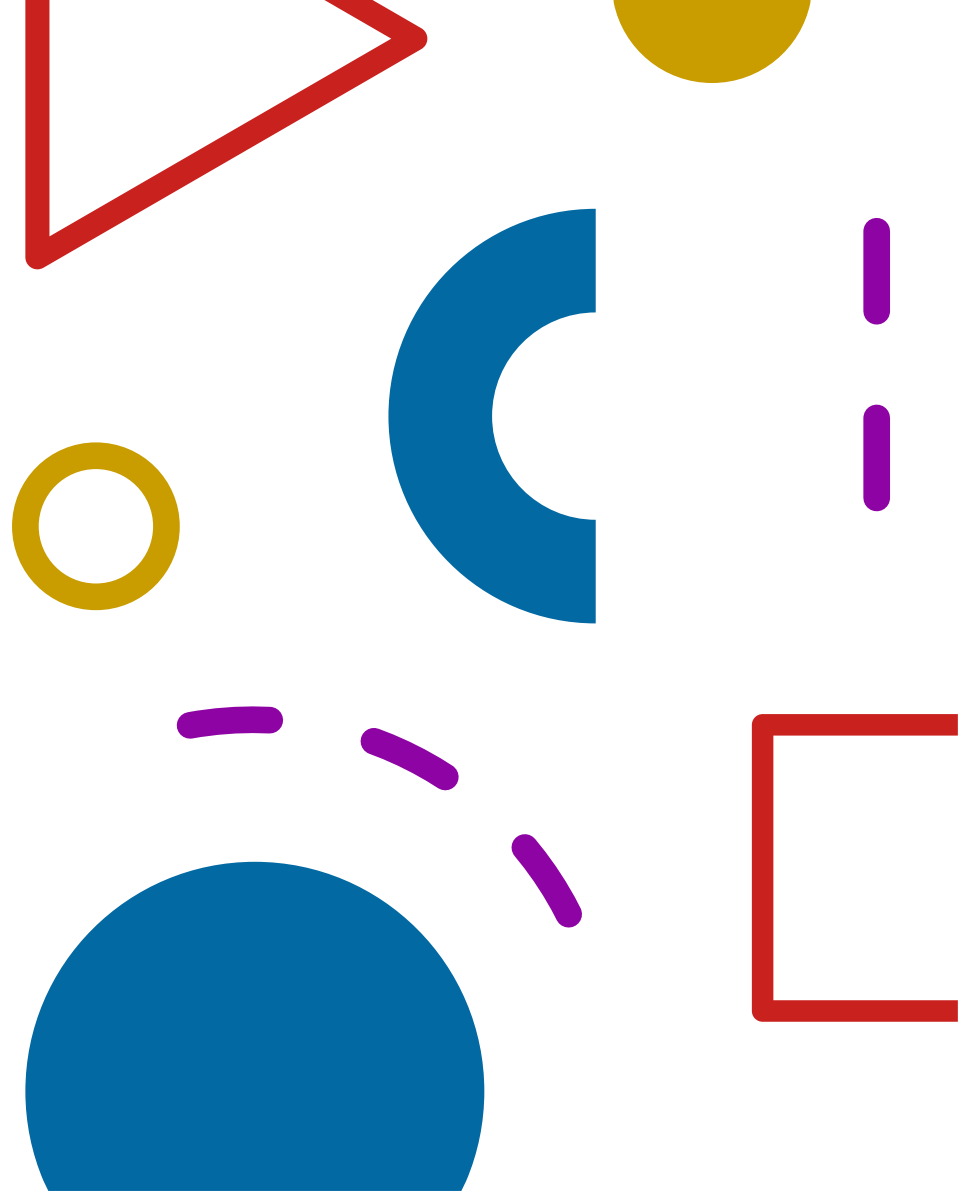
- Versionar el código.
- Versionar los datos.
- Validar los datos.
- Evaluar la calidad de los datos.
- Monitorizar cambios.
- Actualizar modelos.

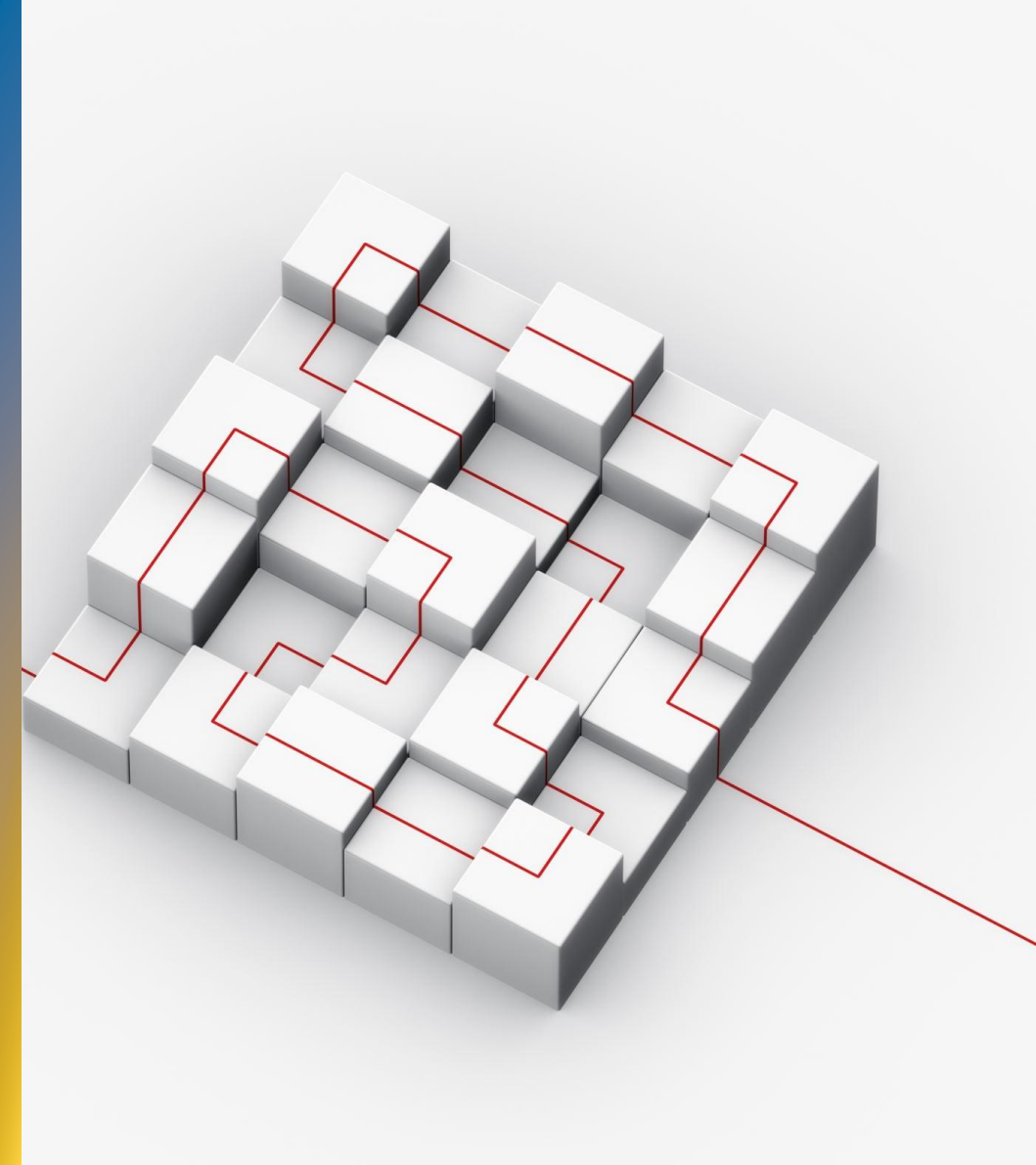
En ML no solo debemos preguntar “¿el código funciona?”, sino también “¿los datos y el modelo siguen siendo adecuados?”.

El desafío de llevar ML a producción

- Los sistemas ML modernos pueden tener cientos de millones o incluso miles de millones de parámetros. Esto genera desafíos:
 -  Infraestructura → modelos grandes requieren mucha memoria y capacidad de cómputo.
 - ⚡ Rendimiento → el modelo debe responder suficientemente rápido.
 -  Edge devices → ejecutar modelos grandes en dispositivos pequeños es difícil.
 -  Monitoreo → detectar cuándo el modelo deja de funcionar correctamente.
 -  Debugging → entender qué salió mal puede ser complejo.
 -  Actualización → los datos y patrones cambian con el tiempo.
- El desafío es conseguir que el sistema sea:
- **Preciso + rápido + escalable + justo + interpretable + mantenible.**

Arquitectura de Machine Learning





Introducción a la arquitectura de ML

- Estructura y principios para diseñar sistemas de ML.
- Integra software, hardware, datos y modelos.
- Permite desarrollar, implementar y administrar modelos.
- Busca sistemas escalables, mantenibles y robustos.

¿Qué es la arquitectura de ML?

La arquitectura de ML proporciona un modelo estructural para construir sistemas capaces de resolver problemas específicos mediante aprendizaje automático.

Incluye:

- Dependencias entre componentes.
- Relaciones y flujos de trabajo.
- Procesos de desarrollo y operación.
- Infraestructura necesaria para los modelos.

Objetivo: lograr un sistema eficiente desde la creación hasta el mantenimiento del modelo.

¿Por qué es importante?

- Una arquitectura adecuada permite:
 - Mejorar los resultados de los modelos.
 - Reducir costos computacionales y operativos.
 - Facilitar la reproducibilidad.
 - Escalar los sistemas ante grandes volúmenes de datos.
 - Mantener y actualizar los modelos con mayor facilidad.
 - Detectar y afrontar problemas como el sesgo y la deriva de datos.

Flujo de Trabajo del ML

1. Ingesta de datos

- Obtención y almacenamiento de los datos necesarios.

2. Preprocesamiento

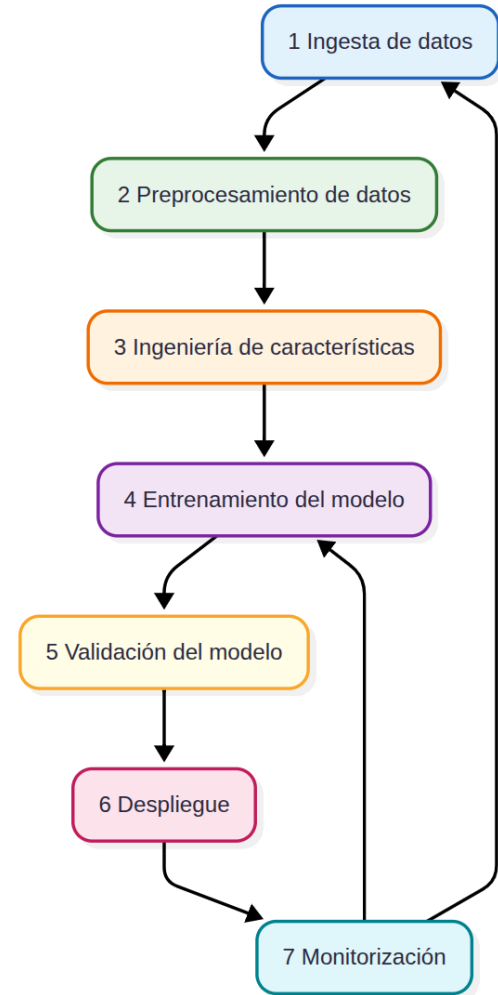
- Limpieza, transformación y preparación de los datos.

3. Ingeniería de características

- Selección o creación de variables útiles para el modelo.

4. Entrenamiento

- Construcción y ajuste del modelo utilizando los datos.



Flujo de Trabajo del ML

5. Validación del modelo

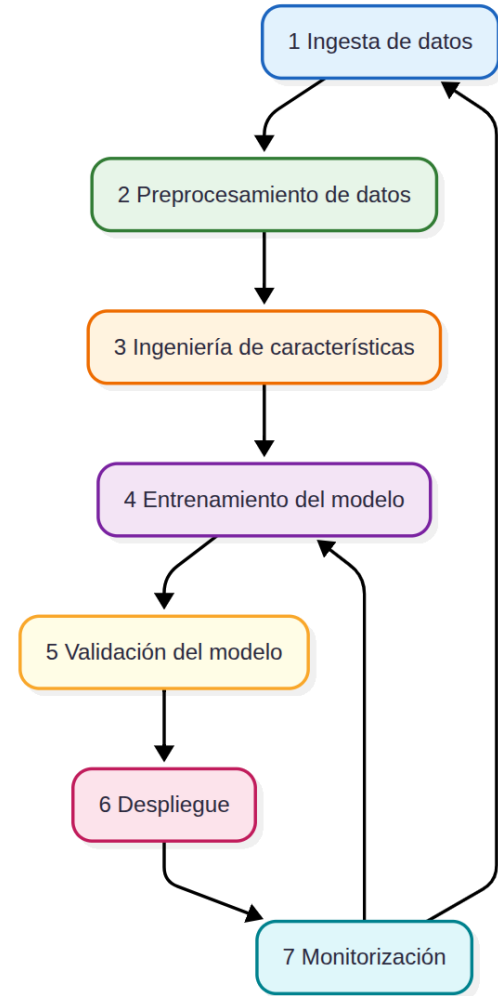
- Evalúa el rendimiento.
- Comprueba si el modelo generaliza correctamente.

6. Despliegue

- Lleva el modelo a un entorno donde pueda utilizarse.
- Puede realizar inferencias por lotes o en tiempo real.

7. Monitorización

- Supervisa el comportamiento del modelo.
- Permite detectar cambios y problemas después de su implementación.

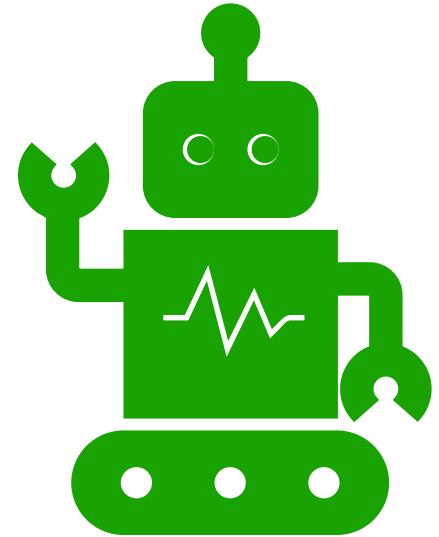


Tecnologías utilizadas

- Las arquitecturas modernas de ML utilizan diferentes herramientas y plataformas:

- **TensorFlow:** desarrollo de modelos de aprendizaje automático.
- **PyTorch:** entrenamiento y desarrollo de modelos.
- **scikit-learn:** algoritmos y herramientas de ML.
- **Computación en la nube:** recursos escalables.
- **Sistemas distribuidos:** procesamiento de grandes cantidades de datos.

- La elección depende de las necesidades y características de cada proyecto.



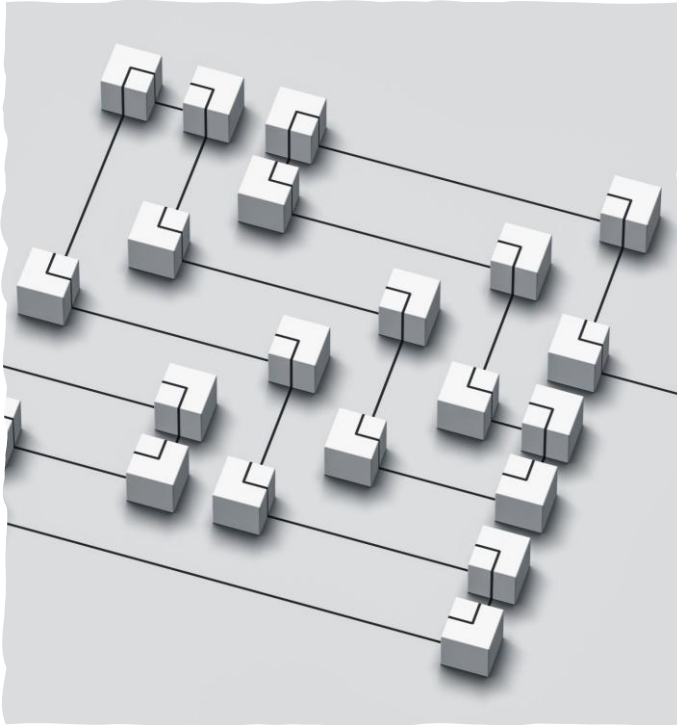
Selección del modelo

- Elegir el modelo adecuado es una decisión fundamental dentro de la arquitectura de ML.
- Una selección apropiada puede favorecer:
 - El rendimiento del sistema.
 - La reproducibilidad.
 - La escalabilidad.
 - La facilidad de mantenimiento.
 - La inferencia por lotes o en tiempo real.
- La arquitectura debe adaptarse al problema que se busca resolver.

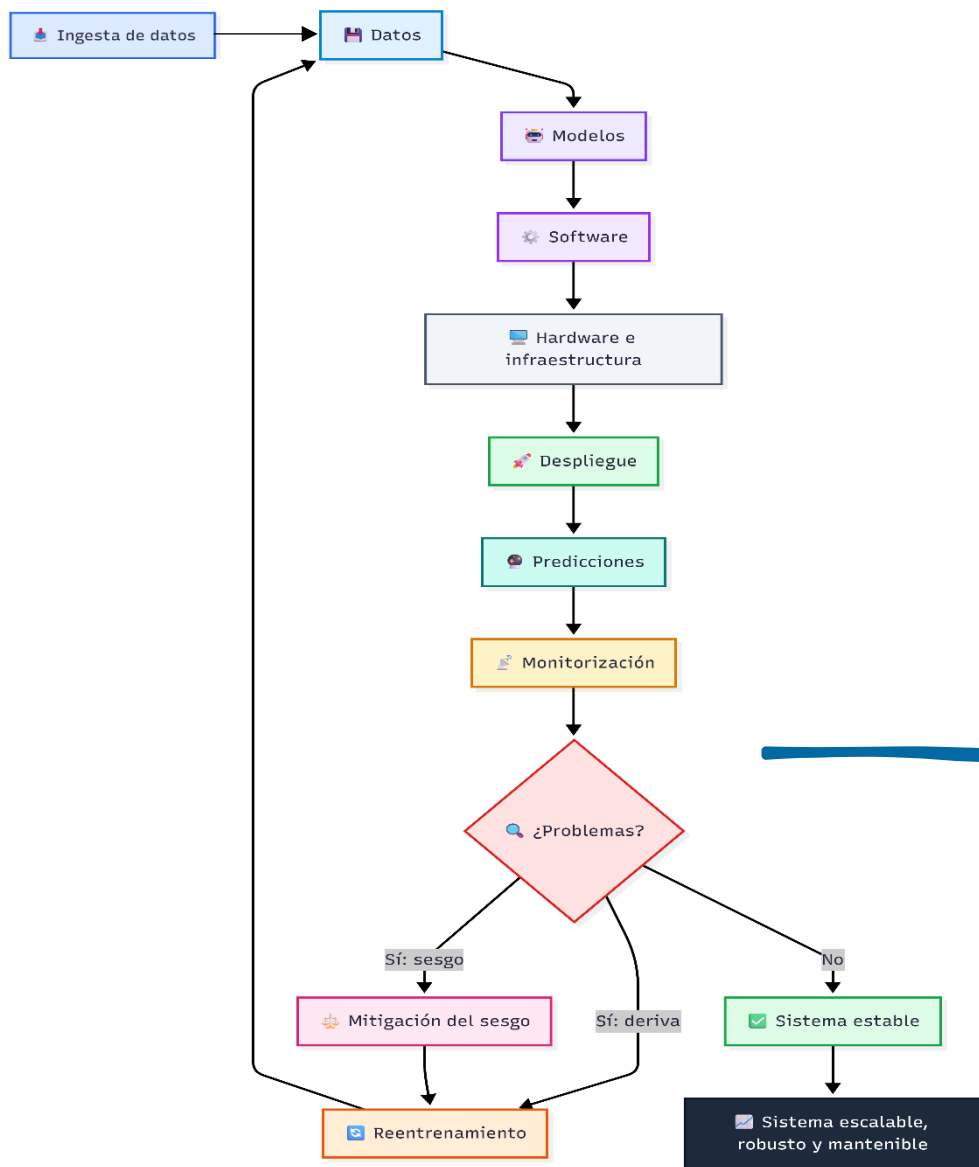
Principales desafíos

- Los sistemas de ML deben afrontar diferentes problemas:
 - **Deriva de datos:** Los datos pueden cambiar con el tiempo y afectar el rendimiento.
 - **Sesgo del modelo:** Los modelos pueden producir resultados sistemáticamente desfavorables debido a sesgos presentes en los datos o en el proceso.
 - **Transparencia:** Es importante comprender y explicar cómo el sistema llega a sus resultados.
 - **Escalabilidad:** La infraestructura debe ser capaz de manejar el crecimiento de datos y usuarios.

Beneficios de la Arquitectura ML



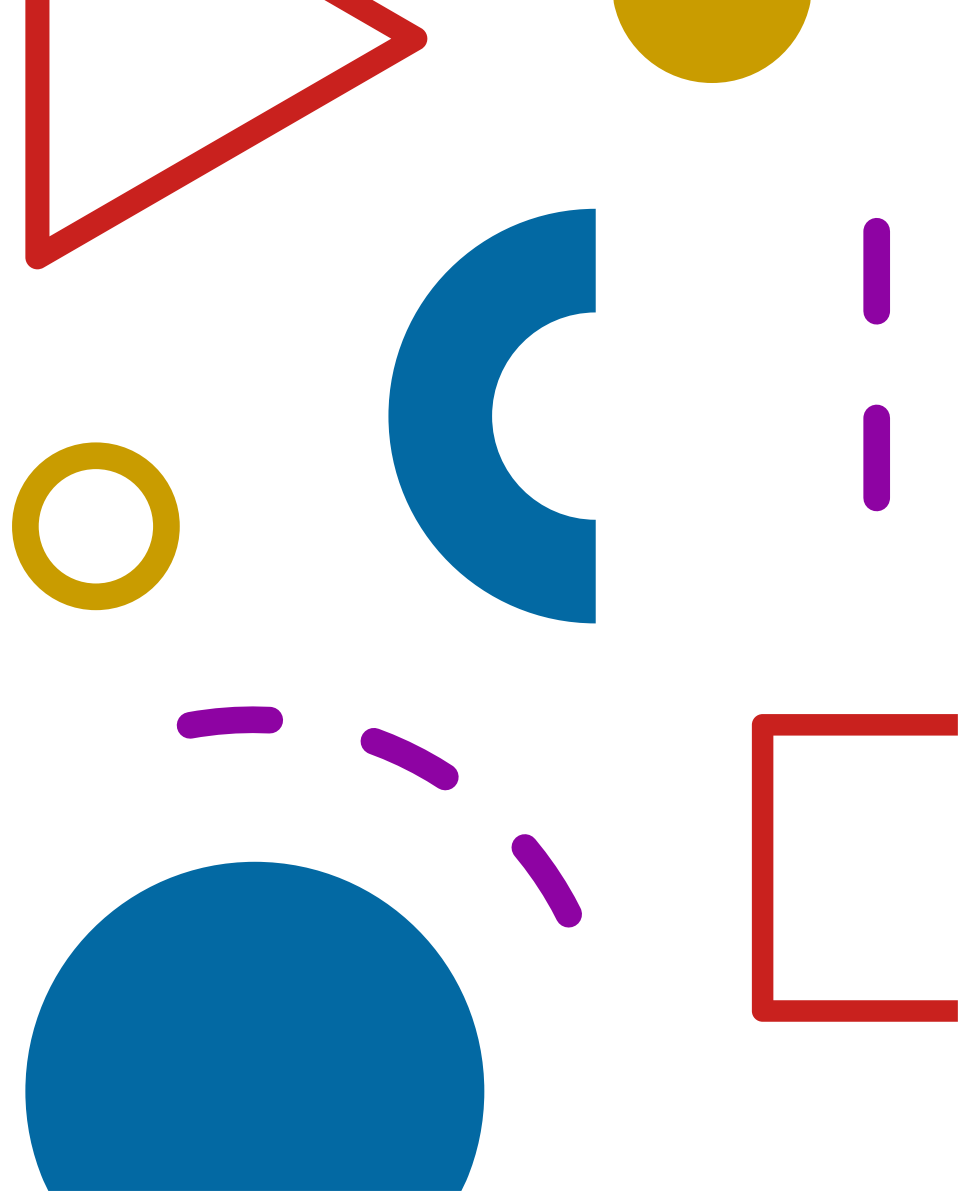
- La arquitectura de ML permite organizar de forma estructurada todo el ciclo de vida de un sistema de aprendizaje automático.
 - Integra datos, modelos, software, hardware e infraestructura.
 - Abarca desde la ingesta de datos hasta la monitorización.
 - Facilita sistemas escalables, robustos y mantenibles.
 - La elección de modelos y tecnologías influye directamente en los resultados.
 - Una buena arquitectura ayuda a enfrentar desafíos como el sesgo y la deriva de datos.



Arquitectura de ML

Ciclo de vida y componentes

Enfoques de Machine Learning



Tipos de Machine Learning

The Three Types of Machine Learning

Supervised Learning

Labeled training data

Labeled Data

Model

Examples:

- Classification
- Regression
- Spam detection
- Price prediction

Unsupervised Learning

Unlabeled data



Examples:

- Clustering
- Dimensionality reduction
- Anomaly detection
- Customer segmentation

Reinforcement Learning

Agent + Environment

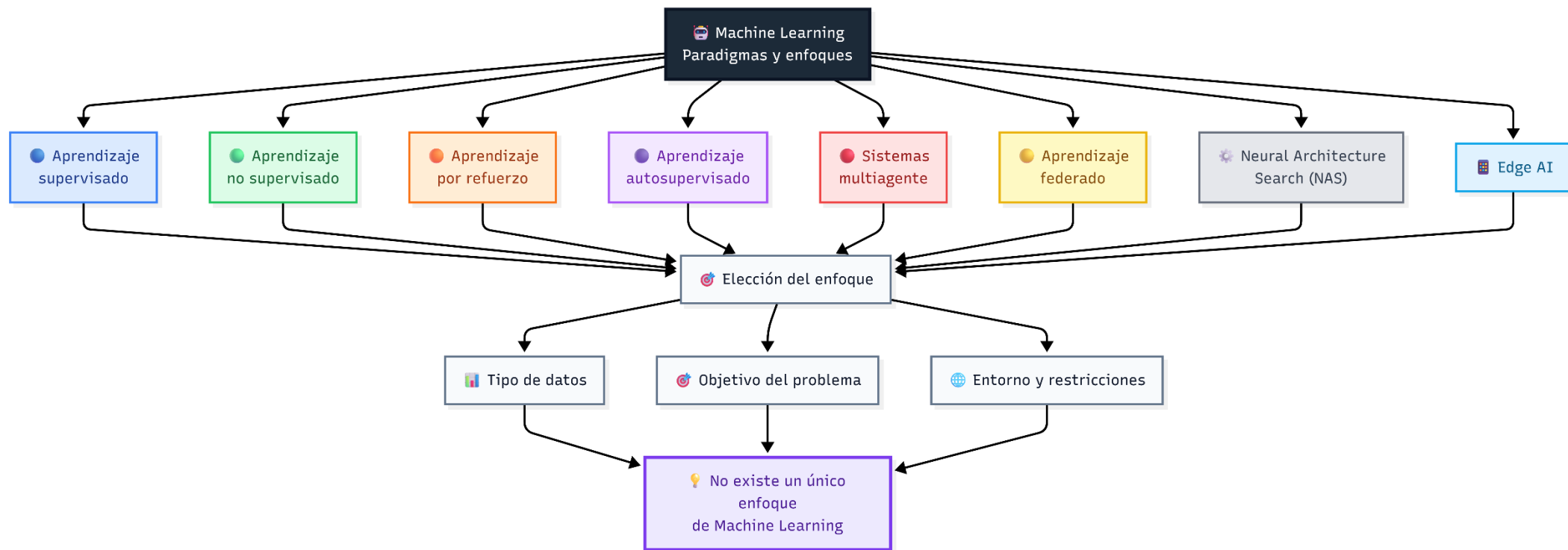
Agent

Reward

Examples:

- Game playing (AlphaGo)
- Robot navigation
- Recommendation
- Autonomous driving

Enfoques de Machine Learning



Clasificación de los enfoques de ML

Enfoques tradicionales



Supervisado → Aprende con datos etiquetados.

No supervisado → Descubre patrones en datos sin etiquetas.

Por refuerzo → Aprende mediante recompensas y penalizaciones.

Enfoques modernos

Autosupervisado → Aprende representaciones a partir de los propios datos.

Multiagente → Múltiples agentes interactúan en un entorno.

Federado → Entrenamiento distribuido sin centralizar necesariamente los datos.

NAS (Neural Architecture Search) → Busca automáticamente arquitecturas de redes neuronales.

Edge AI → Ejecuta modelos directamente en dispositivos periféricos.

Aprendizaje supervisado

- El modelo aprende utilizando ejemplos donde conocemos: Entrada (X) $\rightarrow$ Etiqueta/Salida (Y)
- Ejemplo:
- 🏠 Características de una vivienda $\rightarrow$ Precio
- 📧 Características de un correo $\rightarrow$ Spam / No spam
- 🩺 Información médica $\rightarrow$ Diagnóstico
- Objetivo: Aprender una función: $f(X) \rightarrow Y$ que pueda generalizar correctamente a datos que el modelo nunca ha visto.

Flujo del aprendizaje supervisado

01

1. Preparación de datos

- Limpieza.
- Normalización.
- Ingeniería de características.
- División de los datos.

02

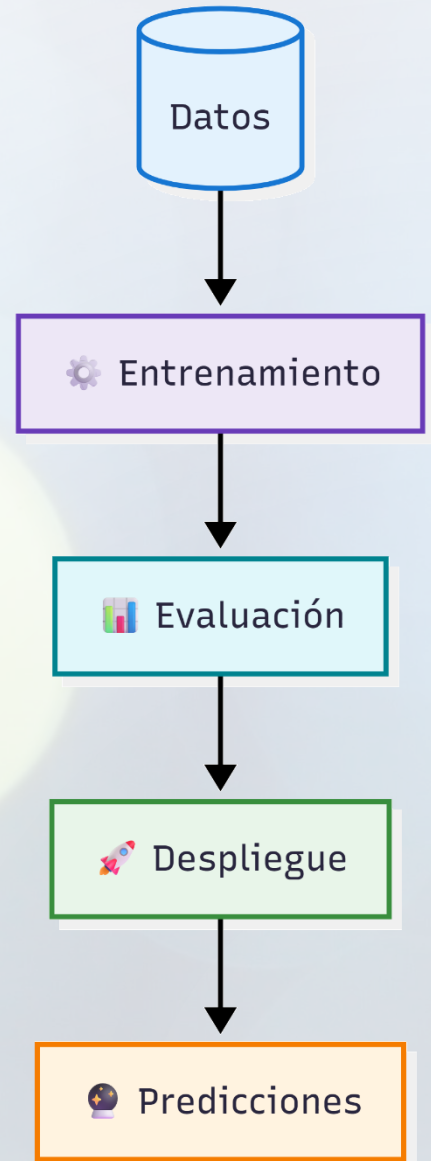
2. Entrenamiento: El modelo ajusta sus parámetros para reducir el error.

03

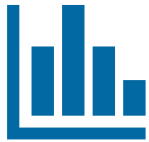
3. Evaluación y validación: Se mide el rendimiento utilizando métricas apropiadas.

04

4. Despliegue: El modelo realiza predicciones sobre nuevos datos.



¿Cómo se evalúa un modelo?



Las métricas dependen del tipo de problema.



Clasificación

Accuracy: proporción de predicciones correctas.

Precision: qué proporción de los positivos predichos realmente son positivos.

Recall: qué proporción de los positivos reales fueron detectados.

Regresión: Mean Squared Error (MSE): mide el error cuadrático promedio.

Validación cruzada: Permite evaluar cómo podría comportarse el modelo con diferentes particiones de los datos.



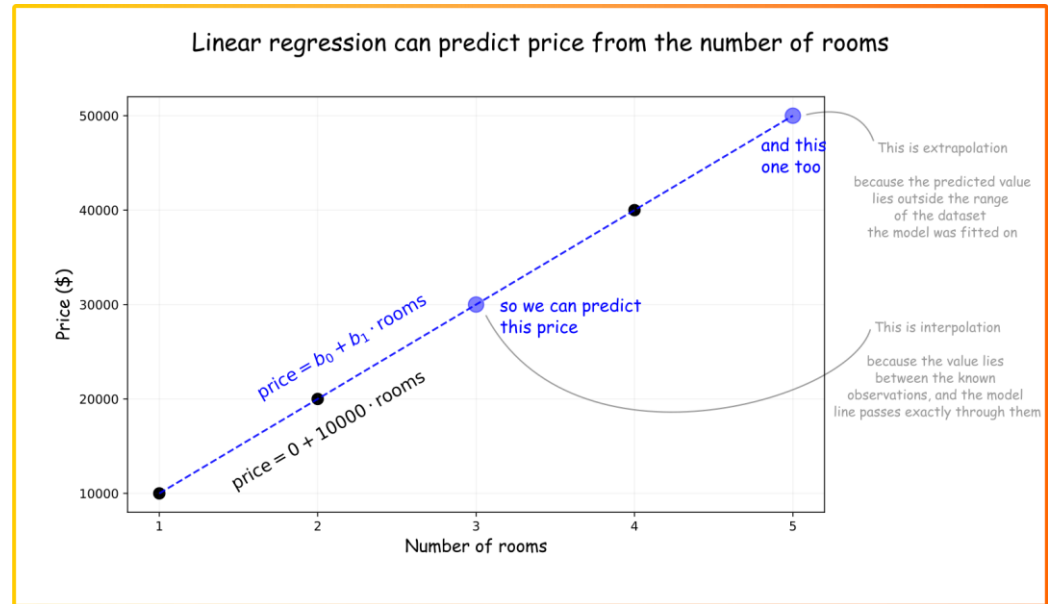
Objetivo: Evitar que el modelo simplemente memorice los datos de entrenamiento.

Algoritmos de aprendizaje supervisado

Algoritmo	Característica principal	Aplicaciones
Regresión lineal	Simple e interpretable	Predicción y análisis
Regresión logística	Probabilidades y clasificación	Spam, enfermedades
Árboles de decisión	Fáciles de interpretar	Riesgo, segmentación
SVM	Maximiza el margen entre clases	Imágenes, escritura
Redes neuronales	Capturan patrones complejos	Voz, lenguaje

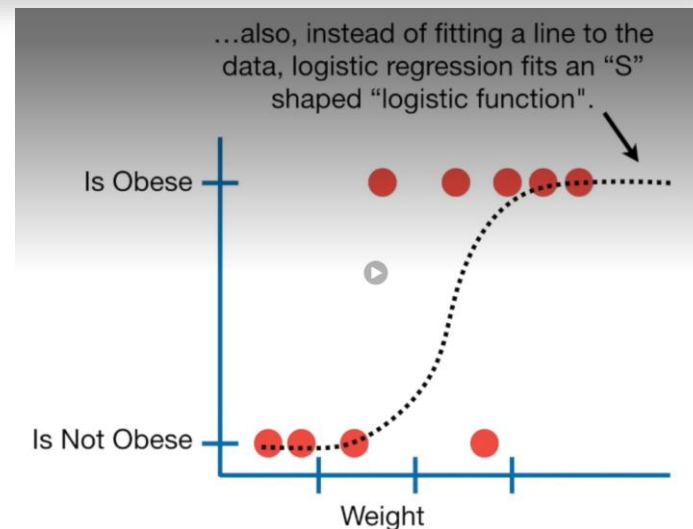
Regresión lineal

- Predice un valor numérico.
- Ejemplos:
- Precio de una vivienda.
- Demanda de un producto.
- Ingresos estimados.
- Salida: valor continuo.



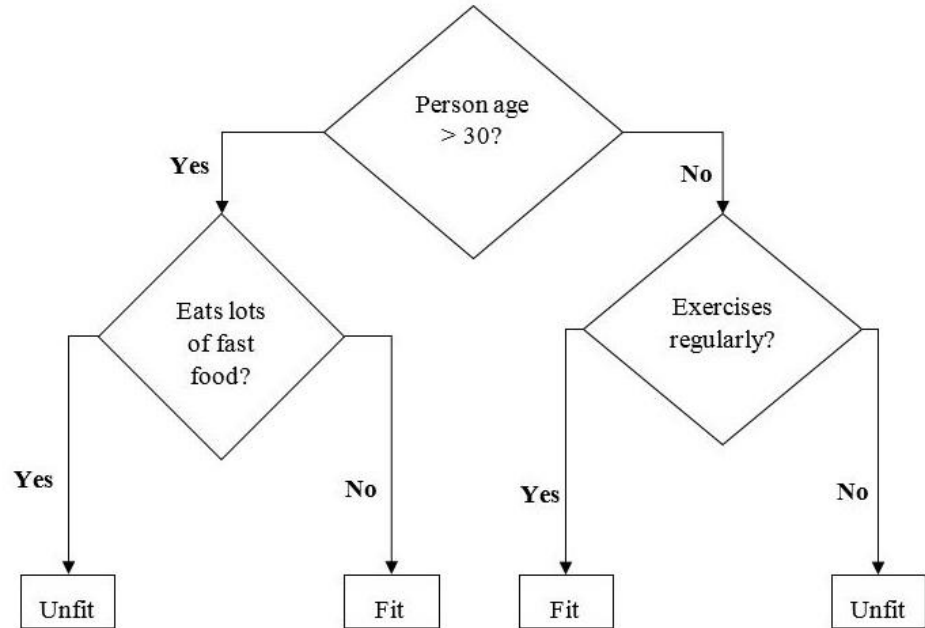
Regresión logística

- Se utiliza principalmente para clasificación.
- Ejemplos:
 1. Spam / no spam.
 2. Enfermo / sano.
 3. Fraude / transacción normal.
- Puede producir una probabilidad asociada a cada clase.



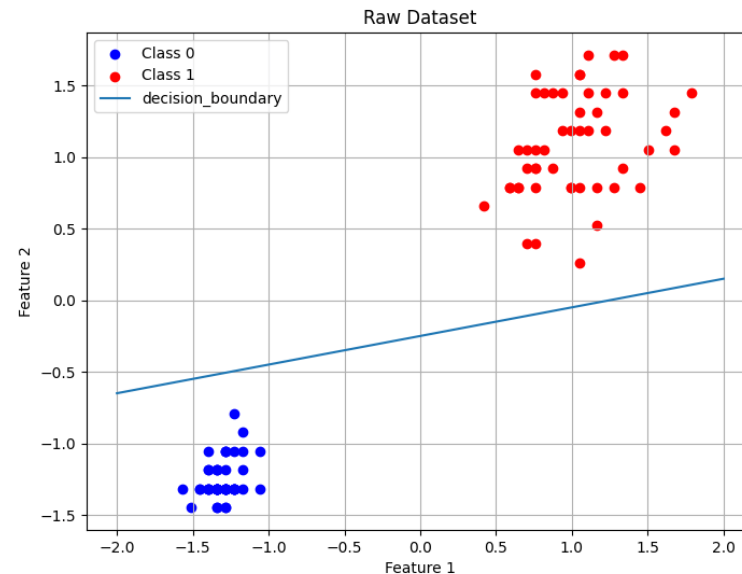
Árboles de decisión

- Toman decisiones mediante una secuencia de condiciones.
- Ventajas:
- Interpretables.
- Fáciles de visualizar.
- Útiles para clasificación y predicción.

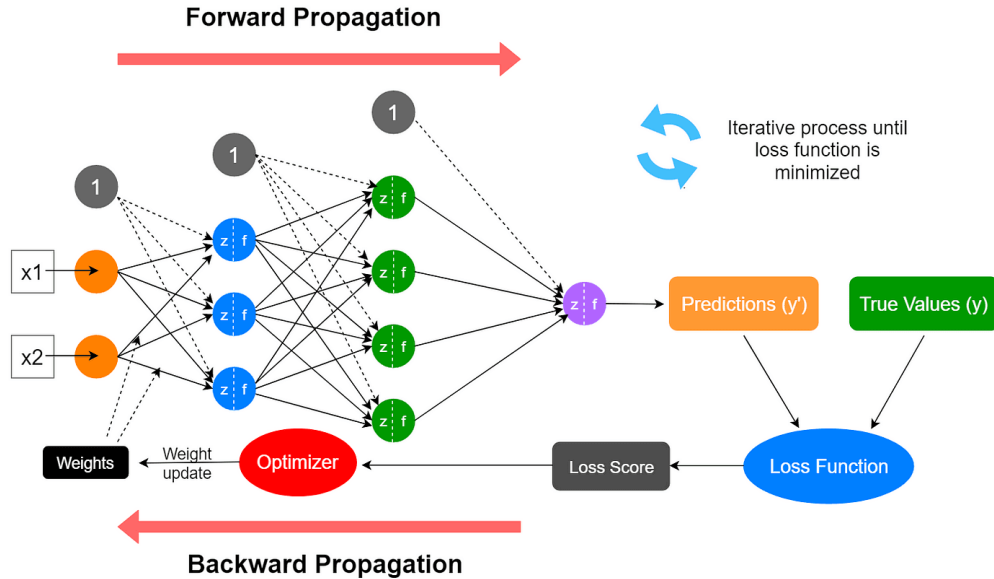


Support Vector Machines (SVM)

- Buscan separar diferentes clases maximizando el margen entre ellas.
- Aplicaciones:
- Clasificación de imágenes.
- Reconocimiento de escritura.
- Clasificación de datos complejos.



Redes Neuronales



Las redes neuronales pueden aprender representaciones y patrones complejos.

Características

-  Gran capacidad para detectar patrones complejos.
-  Útiles con imágenes, audio y lenguaje.
-  Pueden requerir mucho cómputo.
-  Su interpretación puede ser más difícil.

Aplicaciones

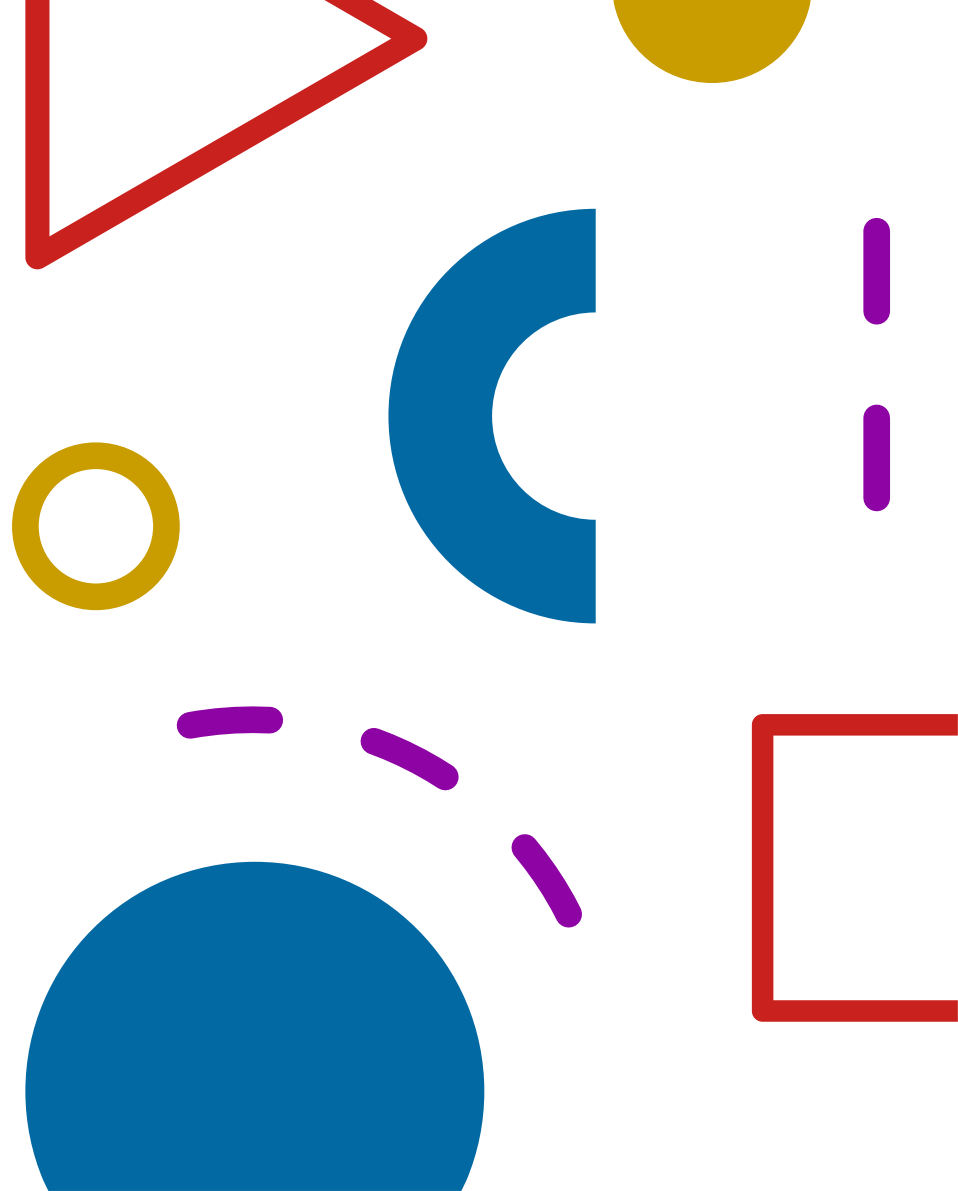
-  Reconocimiento de voz
-  Procesamiento de lenguaje natural
-  Reconocimiento de imágenes
-  Sistemas inteligentes

Tomado de [Alation](#)

¿Qué enfoque elegir?

Enfoque	Datos	Objetivo
Supervisado	Etiquetados	Predecir una salida
No supervisado	Sin etiquetas	Descubrir patrones
Refuerzo	Recompensas	Tomar decisiones
Autosupervisado	Etiquetas generadas del propio dato	Aprender representaciones
Multiagente	Interacción entre agentes	Resolver problemas dinámicos
Federado	Datos distribuidos	Aprender preservando la descentralización
NAS	Datos + búsqueda de arquitecturas	Diseñar modelos automáticamente
Edge AI	Datos en dispositivos	Inferencia cerca del usuario

Flujo de Trabajo Machine Learning





Clasificación de flores

Flor de Iris



- La flor de iris es una de las flores más elegantes y simbólicas del mundo floral.
- Su nombre es adoptado de la diosa griega Iris, mensajera de los dioses y la personificación del arcoíris, lo cual puede darnos alguna pista sobre su riqueza en colores y significados.
- Con su belleza y su historia llena de simbolismo, el iris se ha convertido en una de las flores más populares en la actualidad.

Problema de Transformación Digital

- El mercado extranjero exige que las flores sean entregadas de forma clasificada.
- Para el caso del iris la clasificación se determina por las dimensiones del pétalo y del sépalo
- En la exportadora de flores se ha contratado a gran cantidad de personal para que haga esta clasificación, pero por la cantidad y delicadeza de las flores, se cometen errores humanos y se manipulan las flores.

Concepto del Líder de Transformación Digital

Para evitar la manipulación de las flores en el momento de clasificarlas, sugiere que se instalen cámaras con software de reconocimientos de imágenes.

Una vez reconocida la flor iris, aplicar un **modelo de machine learning que determine la clasificación de la especie de flor.**

Determinada la especie se embala la flor en el canal correspondiente.





Iris Versicolor



Iris Setosa



Iris Virginica

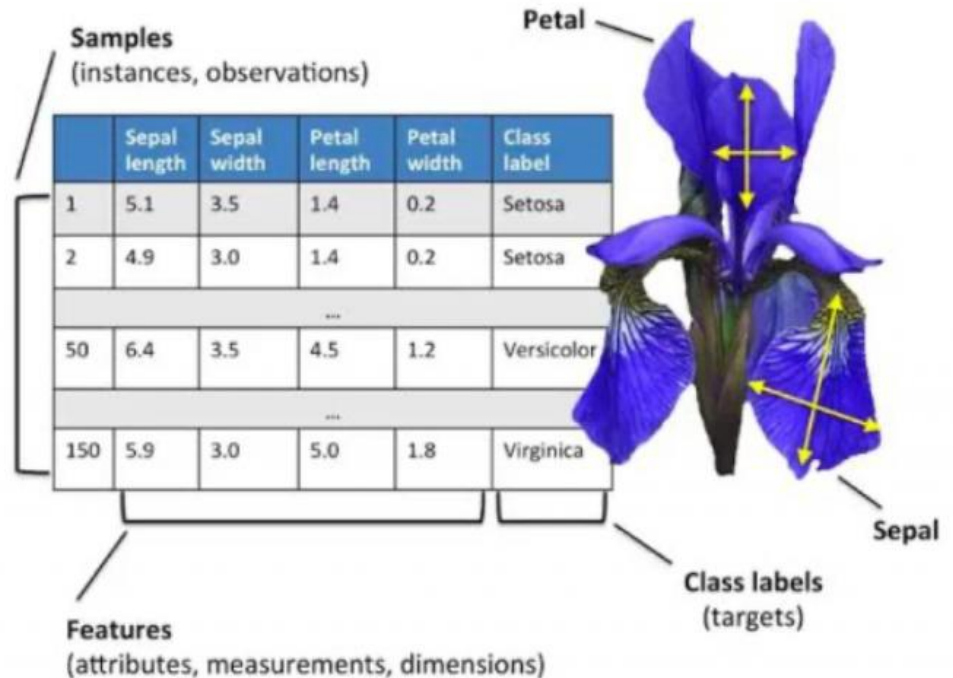
Tomado de [Guvi HCL](#)

Especies de Flor Iris

- **Iris Setosa** (0): Esta especie se distingue claramente de las otras dos especies de Iris, por lo que puede considerarse como el "modo fácil" de este conjunto de datos para un clasificador.
- **Iris Versicolor** (1): Esta especie es muy similar a Virginica, lo que significa que es más difícil distinguirlas. ¡Ahí radica la verdadera dificultad!
- **Iris Virginica** (2): La tercera especie que es más probable que se confunda con Versicolor basándose únicamente en las medidas de dimensiones.

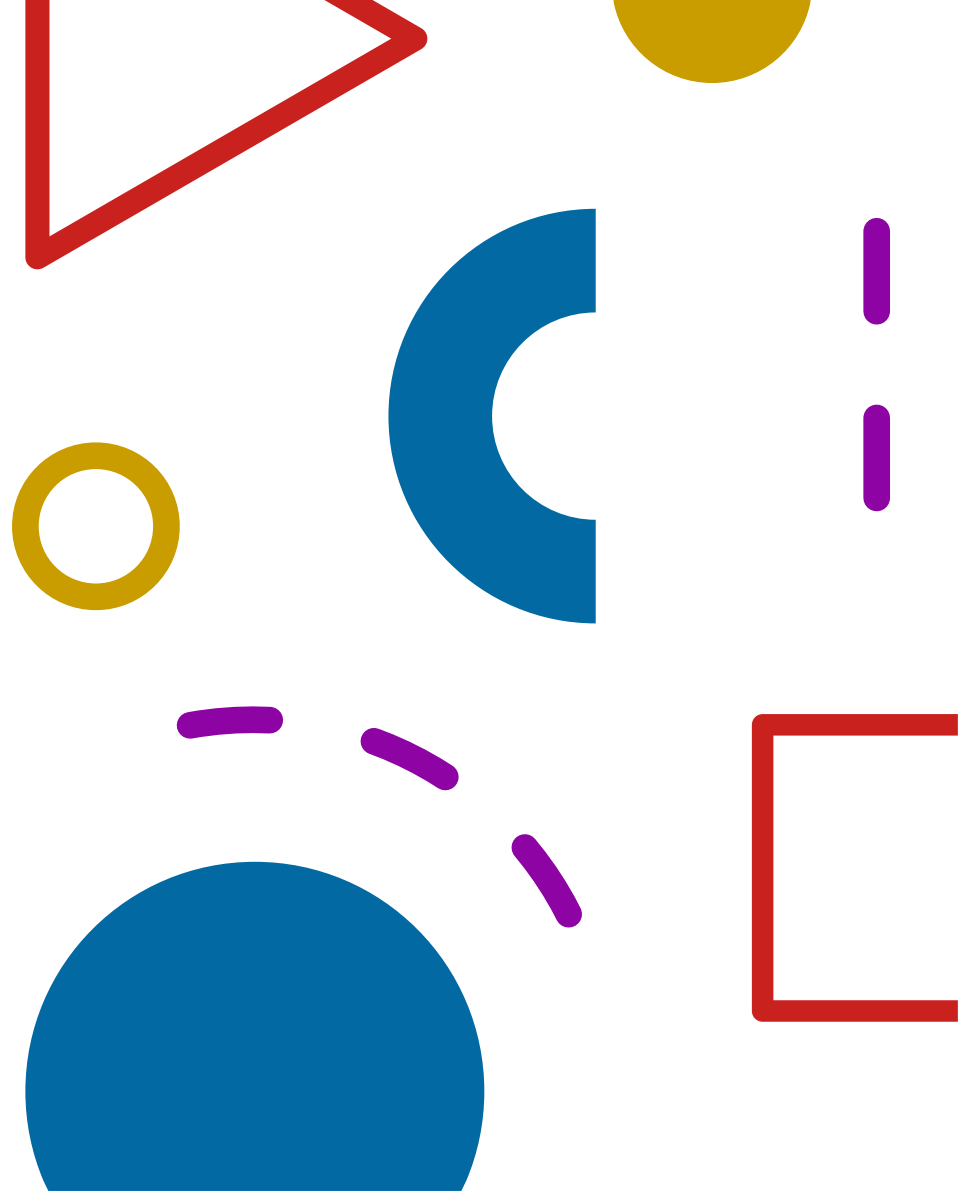
Características de la Flor Iris

- Longitud del sépalo (cm): El sépalo es la parte de la flor que protege el brote antes de florecer.
- Ancho del sépalo (cm)
- Longitud del pétalo (cm): El pétalo es la parte muy colorida de la flor
- Ancho del pétalo (cm)



Tomado de [Guvi HCL](#)

Configuración del ambiente Python



Descargar Instalador de Python

The screenshot shows the Python.org homepage. The navigation bar includes links for About, Downloads, Documentation, Community, Success Stories, News, and Events. A search bar and a 'Donate' button are also present. The 'Downloads' menu is open, showing options for All releases, Source code, Windows, macOS, Android, iOS, Other Platforms, License, and Alternative Implementations. The 'Python Source' section is highlighted, showing the 'Python 3.14.7' button. A green arrow points to this button. Below the 'Python 3.14.7' button, there is a link to 'View the full list of downloads.' and a 'Download for Windows' button. The 'Download for macOS' button is also visible. The 'Get Started' section is visible on the left, and the 'Download' section is visible on the right.

python™

Donate Search GO Socialize

About Downloads Documentation Community Success Stories News Events

Simple arithmetic
>>> 1 / 2
0.5
>>> 2 ** 3
8
>>> 17 / 3 #
5.666666666666667
>>> 17 // 3 #
5

All releases
Source code
Windows
macOS
Android
iOS
Other Platforms
License
Alternative Implementations

Python Source

Python 3.14.7

Not the OS you are looking for? Python can be used on many operating systems and environments.
View the full list of downloads.

Download for Windows

Python install manager

Or get the standalone installer for Python 3.14.7

Not the OS you are looking for? Python can be used on many operating systems and environments.
View the full list of downloads.

Download for macOS

Python 3.14.7

Not the OS you are looking for? Python can be used on many operating systems and environments.
View the full list of downloads.

Get Started

Whether you're new to programming or an experienced developer, it's easy to learn and use Python.

Download

Python source code and binaries are available for download in many versions!

Jobs

Looking for work or have a Python related position that you're trying to hire for? Our **relaunched** community-run job board is the

Tomado de [Python.org](https://python.org)

Marcar Add python.exe to PATH



Descargar Antigravity IDE

 Google Antigravity

Products ▾

Use Cases ▾

Pricing

Enterprise

Resources ▾

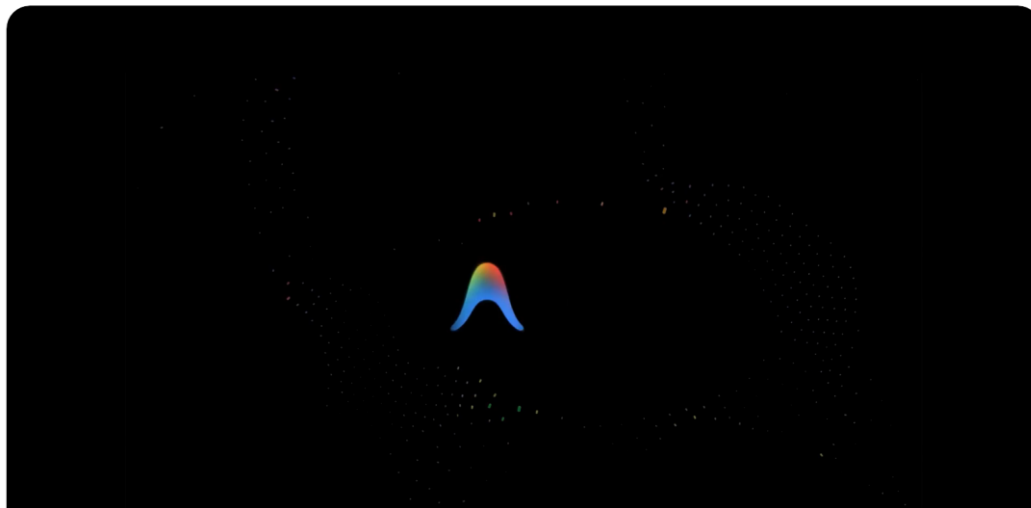


Download

Antigravity IDE

Google Antigravity's Editor view offers tab autocompletion, natural language code commands, and a configurable, and context-aware configurable agent.

Download

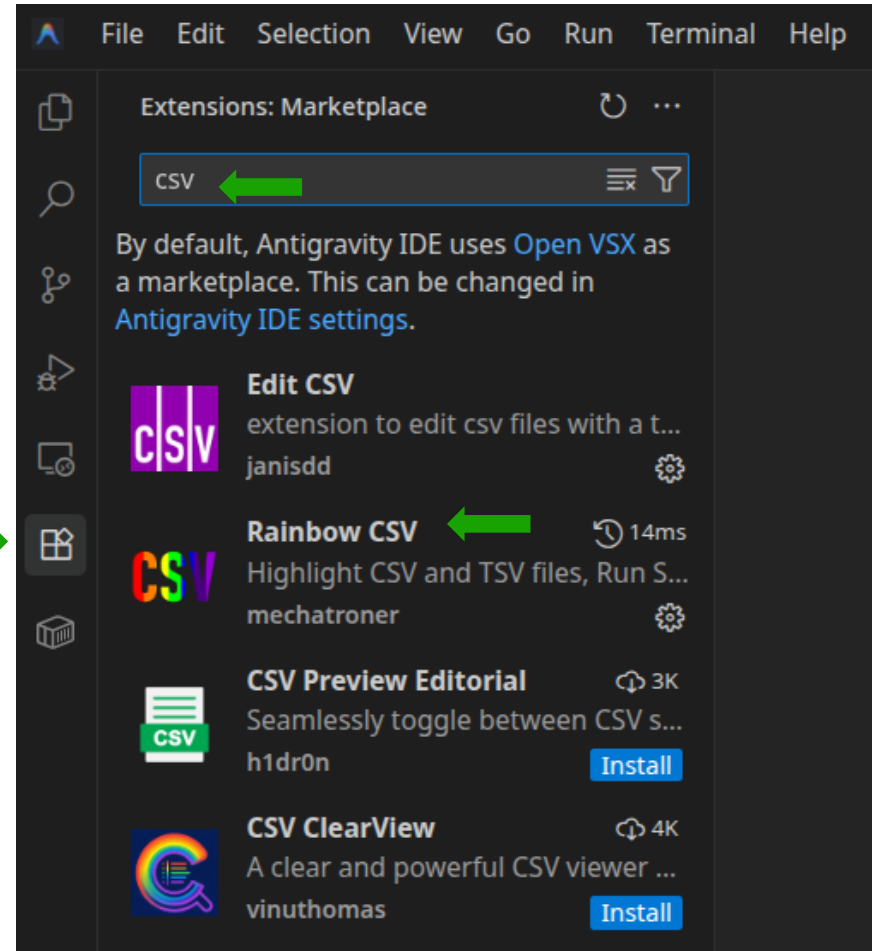


Tomado de [Google Antigravity IDE](#)

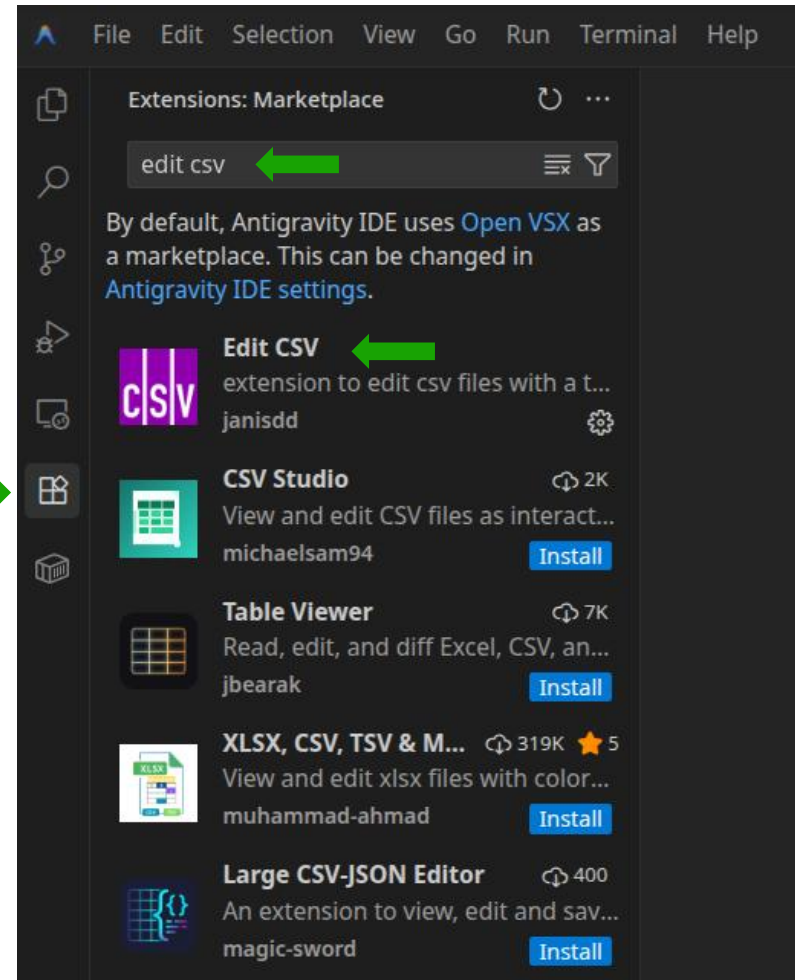
Instalar paquetes de Python (pip)

```
$ pip install pandas
Defauling to user installation because normal site-packages is not writeable
Requirement already satisfied: pandas in ~/.local/lib/python3.14/site-packages (2.3.3)
Requirement already satisfied: numpy ≥ 1.26.0 in ~/.local/lib/python3.14/site-packages (from pandas) (2.2.6)
Requirement already satisfied: python-dateutil ≥ 2.8.2 in ~/.local/lib/python3.14/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz ≥ 2020.1 in ~/.local/lib/python3.14/site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata ≥ 2022.7 in ~/.local/lib/python3.14/site-packages (from pandas) (2025.2)
Requirement already satisfied: six ≥ 1.5 in ~/.local/lib/python3.14/site-packages (from python-dateutil ≥ 2.8.2 → pandas) (1.17.0)
$
```

Extensión CSV



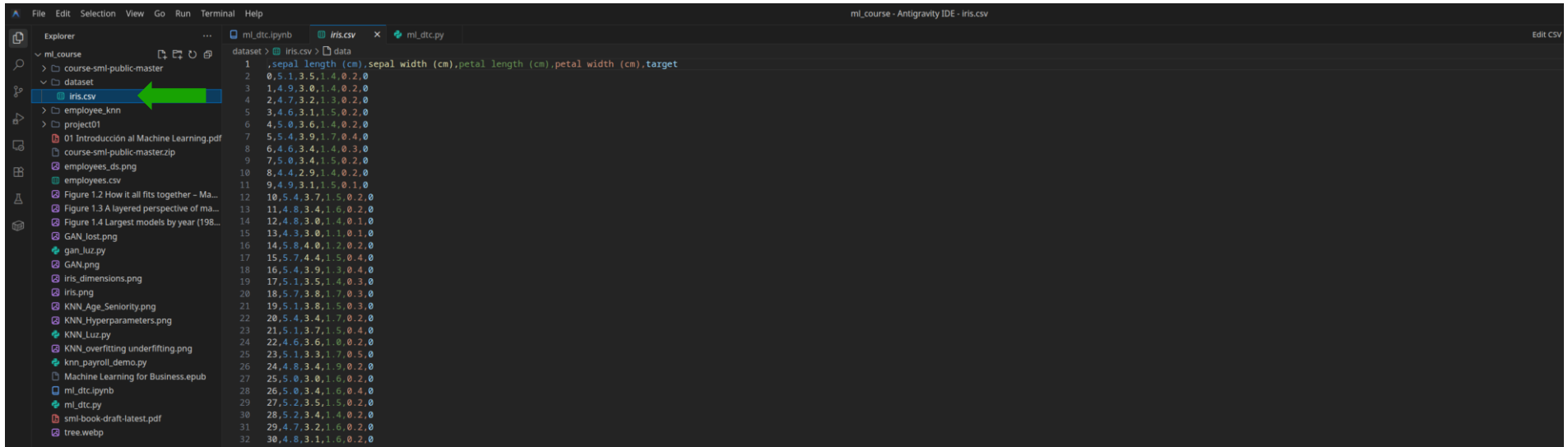
Extensión edit csv



Abrir directorio de trabajo



Mostrar un CSV

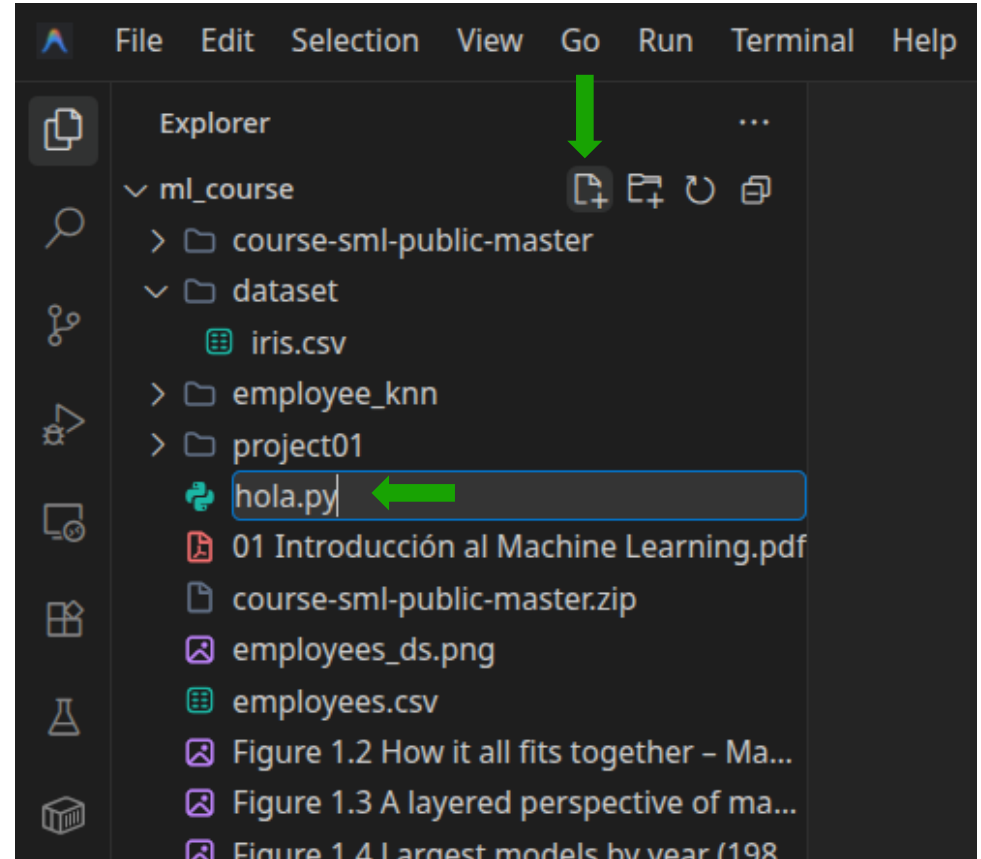


Edit CSV

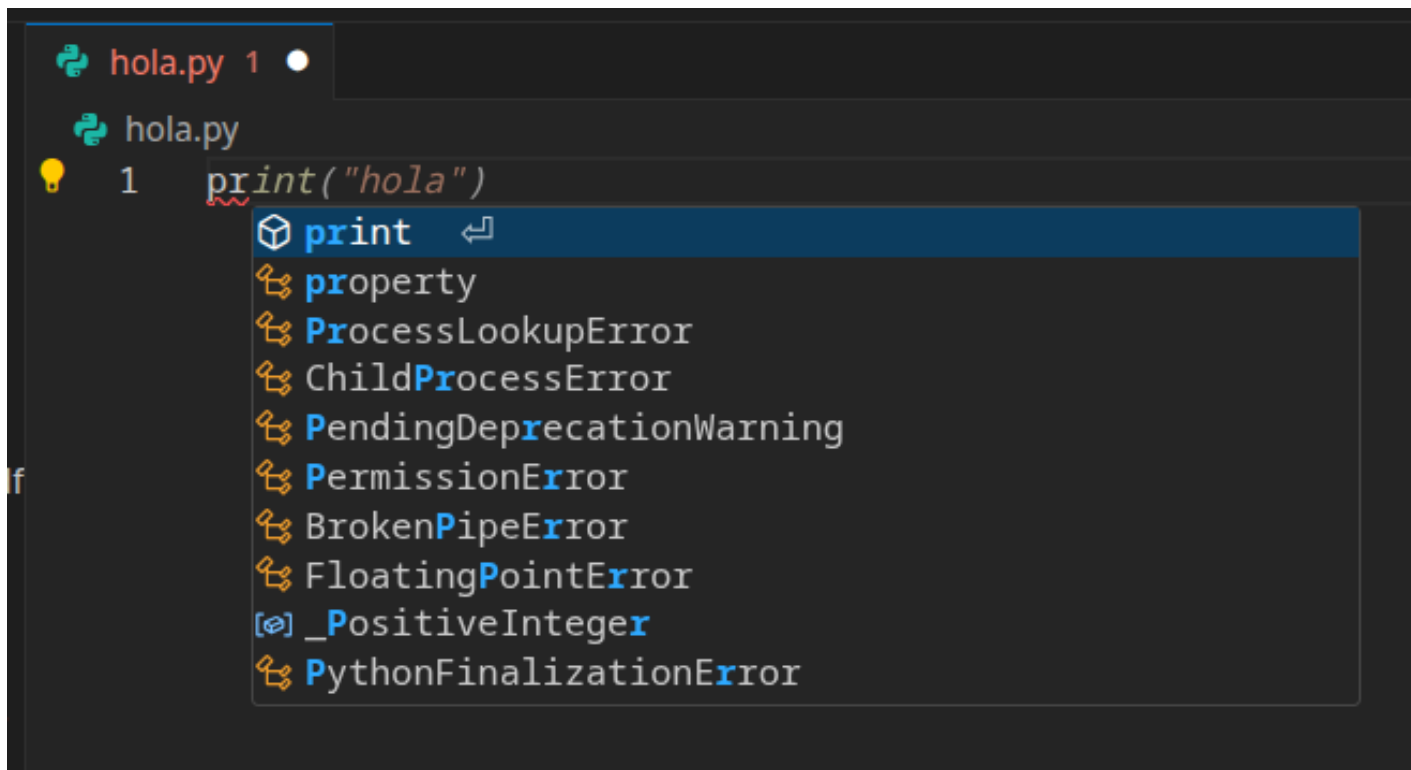
The screenshot shows a JupyterLab interface with the 'CSV edit iris.csv' window open. The window displays a table with 7 columns: column 1, column 2, column 3, column 4, column 5, column 6, and column 7. The data is the Iris dataset, with rows numbered 1 to 21. The first row is highlighted in blue. The interface includes a menu bar (File, Edit, Selection, View, Go, Run, Terminal, Help), a toolbar with icons for file operations, and a sidebar showing the file explorer with a list of files and folders.

	column 1	column 2	column 3	column 4	column 5	column 6	column 7
1		sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	
2	0	5.1	3.5	1.4	0.2	0	
3	1	4.9	3.0	1.4	0.2	0	
4	2	4.7	3.2	1.3	0.2	0	
5	3	4.6	3.1	1.5	0.2	0	
6	4	5.0	3.6	1.4	0.2	0	
7	5	5.4	3.9	1.7	0.4	0	
8	6	4.6	3.4	1.4	0.3	0	
9	7	5.0	3.4	1.5	0.2	0	
10	8	4.4	2.9	1.4	0.2	0	
11	9	4.9	3.1	1.5	0.1	0	
12	10	5.4	3.7	1.5	0.2	0	
13	11	4.8	3.4	1.6	0.2	0	
14	12	4.8	3.0	1.4	0.1	0	
15	13	4.3	3.0	1.1	0.1	0	
16	14	5.8	4.0	1.2	0.2	0	
17	15	5.7	4.4	1.5	0.4	0	
18	16	5.4	3.9	1.3	0.4	0	
19	17	5.1	3.5	1.4	0.3	0	
20	18	5.7	3.8	1.7	0.3	0	
21	19	5.1	2.8	1.5	0.2	0	

Crear un nuevo archivo



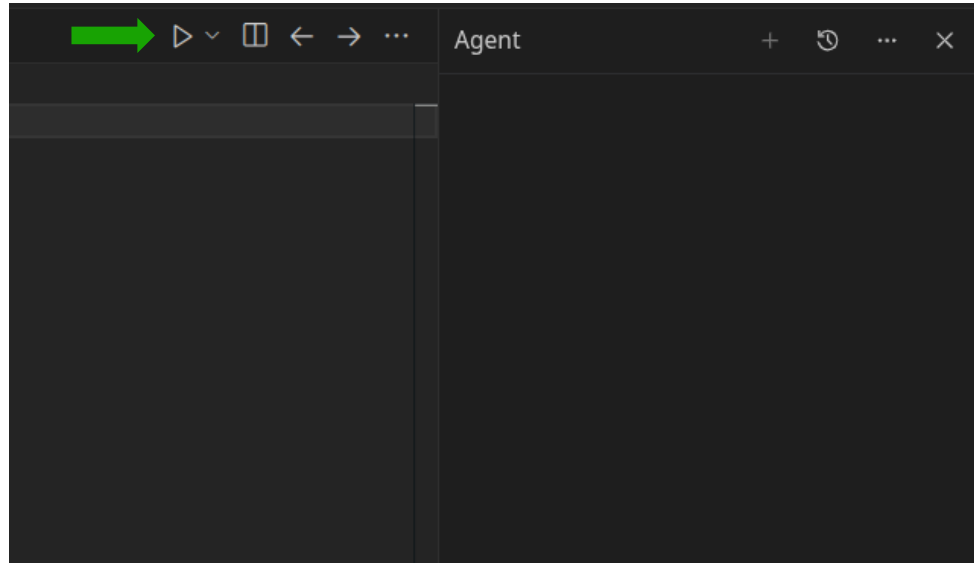
Digitar código



```
hola.py 1 ●  
hola.py  
1 print("hola")
```

- print
- property
- ProcessLookupError
- ChildProcessError
- PendingDeprecationWarning
- PermissionError
- BrokenPipeError
- FloatingPointError
- _PositiveInteger
- PythonFinalizationError

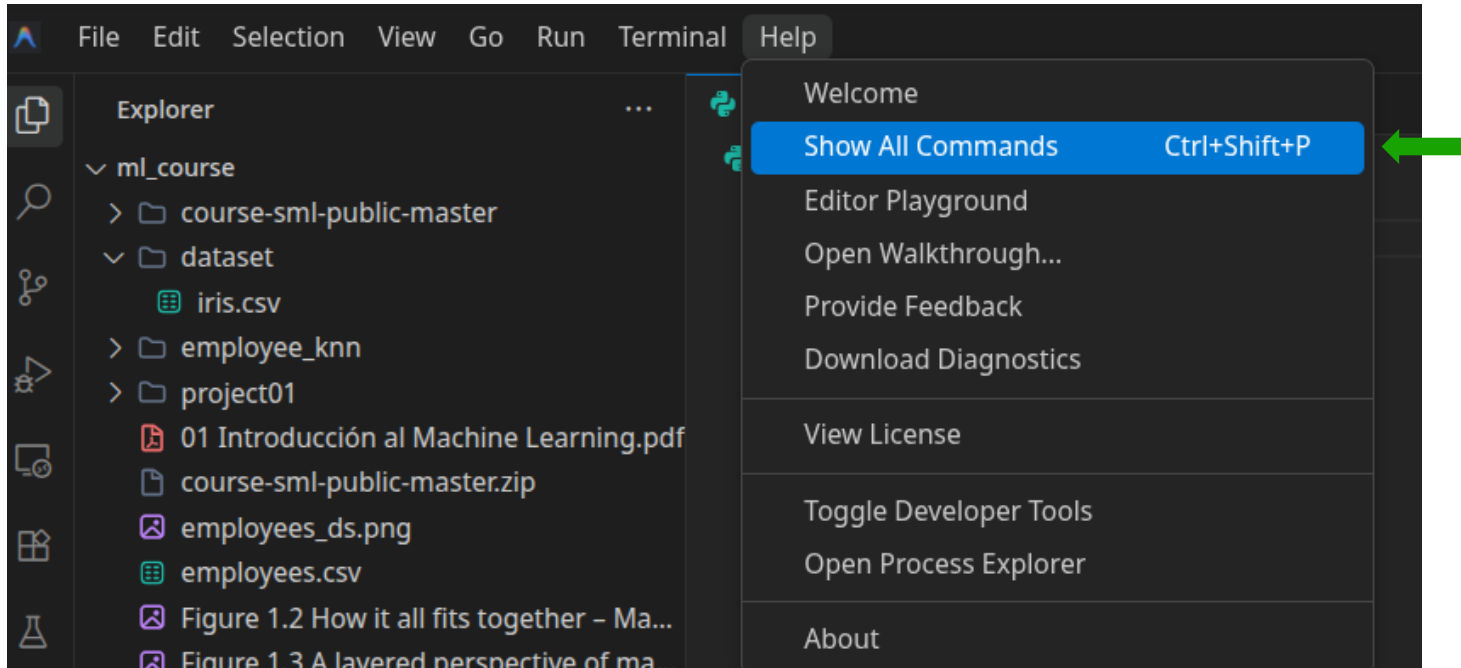
Ejecutar el script de Python



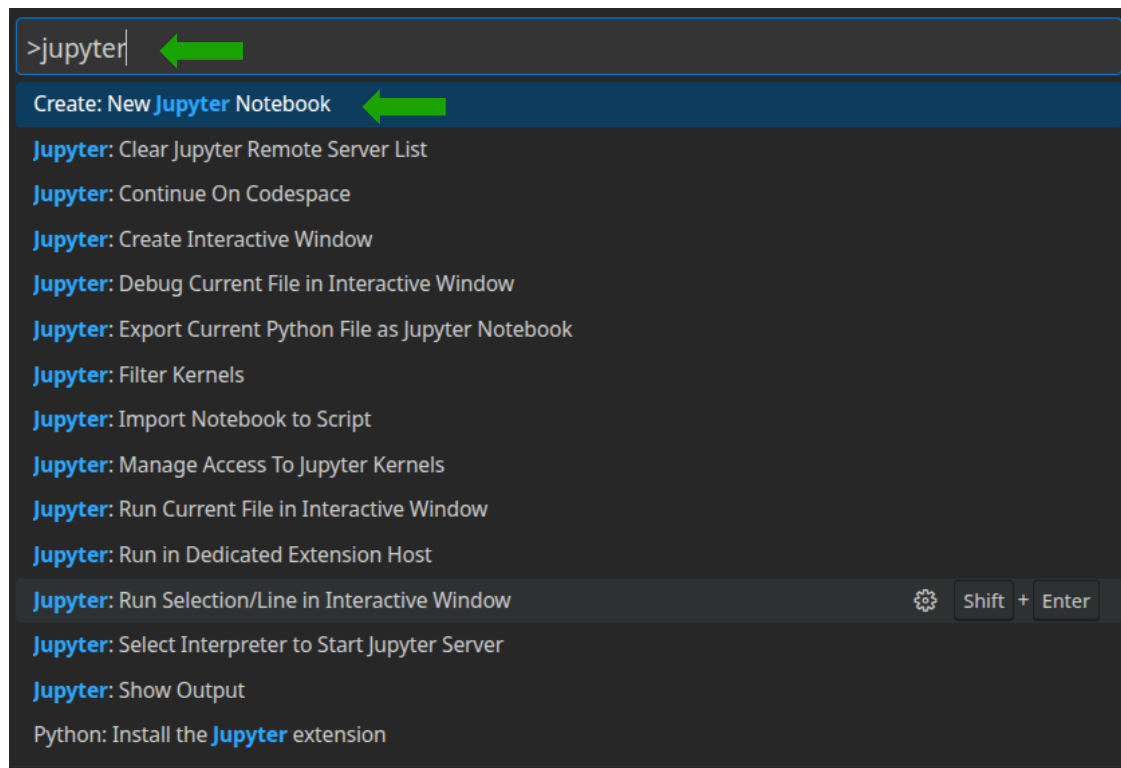
```
Problems  Output  Debug Console  Terminal  Ports  Jupyter

• $ /home/jf/.pyenv/versions/3.11.8/envs/.venv/bin/python "/home/jf/Documentos/personal/family/Luz/u/postgraduate/machine learning/ml_course/hola.py"
  Hola mundo
○ $
```

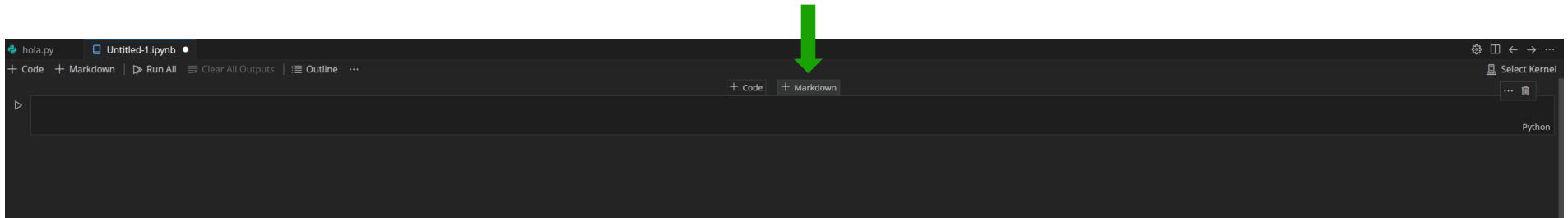
Crear un cuaderno Jupyter



Jupyter Notebook



Celda Markdown



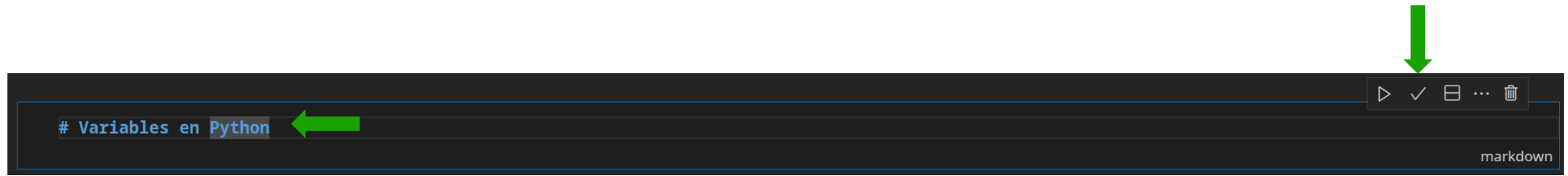
Markdown commands

Markdown	HTML	Resultado
# Encabezado nivel 1	Encabezado nivel 1	Encabezado nivel 1
## Encabezado nivel 2	Encabezado nivel 2	Encabezado nivel 2
### Encabezado nivel 3	Encabezado nivel 3	Encabezado nivel 3
#### Encabezado nivel 4	Encabezado nivel 4	Encabezado nivel 4
##### Encabezado nivel 5	Encabezado nivel 5	Encabezado nivel 5
##### Encabezado nivel 6	Encabezado nivel 6	Encabezado nivel 6
Encabezado nivel 1 =====	Encabezado nivel 1	Encabezado nivel 1

Markdown commands

Markdown	HTML	Resultado
Texto en **negrita** .	Texto en negrita.	Texto en negrita .
Texto en __negrita__ .	Texto en negrita.	Texto en negrita .
Texto en <i>*cursiva*</i> .	Texto en cursiva.	Texto en <i>cursiva</i> .
Texto en <i>_cursiva_</i> .	Texto en cursiva.	Texto en <i>cursiva</i> .
Texto en ***negrita y cursiva***	Texto en negrita y cursiva	Texto en <i>negrita y cursiva</i> .
Texto en ___negrita y cursiva___	Texto en negrita y cursiva	Texto en <i>negrita y cursiva</i> .
Usa el comando `cd`	Usa el comando <code>cd</code>	Usa el comando <code>cd</code>
[Python](https://python.org)	Python	https://python.org
![Santoto](/img/santoto.jpg "Santoto")		
1. Primer elemento 2. Segundo elemento	 - Primer elemento - Segundo elemento 	1. Primer elemento 2. Segundo elemento

Escribir un título nivel 1



```
# Variables en Python
```

markdown

✓ Variables en Python

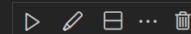
Celda de código Python

Variables en Python

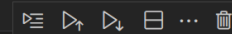


+ Code

+ Markdown



Variables en Python



```
pet_name = "Milo"  
age = 6  
has_vaccines = True  
weight = 8.2
```

```
print(f"Mi mascota se llama {pet_name}, tiene {age} años, tiene vacunas? {has_vaccines} y pesa {weight} kilos")
```

[3] ✓ 0.0s

Python

... Mi mascota se llama Milo, tiene 6 años, tiene vacunas? True y pesa 8.2 kilos



Referencias

- Geertsema, Paul (2023). *Machine Learning for Managers. First published by Routledge*. Taylor & Francis Group, an informa business.
 - Hardy, J. (2025). *Intelligent Automation in Digital Transformation Strategy*. Apress Media LLC. ISBN-13: 979-8-8688-1425-9.
 - Herbert, B. (2026). *Machine Learning with Python*.
 - Reza, M. (2026). *Mastering Machine Learning Architecture and Solutions: From Design to Deployment*. Apress Media LLC. ISBN-13: 979-8-8688-2527-9.
-